

Réf : T1-371-453-455



Université Chouaib Doukkali
École Nationale des Sciences Appliquées d'El Jadida
Département Télécommunications, Réseaux et Informatique



Projet de Fin d'Année

Filière : **ISIC**

Niveau : **2^{ème} Année**

Sujet :

**Développement d'un prototype multicateur pour la surveillance
de l'état de santé et de la vigilance du conducteur**

Réalisé par :

Rajae Elkamili
Safaa Ouard
Houda Riad

Encadré par :

Prof. Asmaa El Hannani

Présenté le 18/05/2026 devant le jury composé de :

Prof. Asmaa EL HANNANI, Professeure à l'ENSAJ

Prof. Ikhlasse HAMZAOUI, Professeure à l'ENSAJ

Année Universitaire : 2025/2026

Résumé

La surveillance du conducteur représente un enjeu majeur de sécurité routière, notamment pour les conducteurs professionnels exposés à la fatigue et aux distractions sur de longues distances. Si des systèmes avancés d'assistance à la conduite (ADAS) existent dans les véhicules haut de gamme, ils demeurent inaccessibles pour la grande majorité des poids lourds en circulation au Maroc, faute de solutions économiques et adaptées au contexte local.

Ce projet de fin d'année propose la conception et le développement d'un prototype multisensoriel de surveillance du conducteur, baptisé DMS DATA COLLECTOR. L'objectif principal est de construire un pipeline de collecte, de synchronisation et de structuration de données hétérogènes issues de quatre dispositifs : un casque EEG (NeuroSky MindWave Mobile 2), un bracelet connecté (iSo Tech), une caméra 3D (ORBEC Astra Pro) et un simulateur de conduite (Logitech G29 Driving Force).

La synchronisation temporelle est assurée par un mécanisme d'horloge logicielle partagée (SyncClock), basé sur `time.perf_counter()` de Python, garantissant une précision inférieure à 1 milliseconde sans dépendance externe. Le stockage adopte une architecture hybride à trois couches : SQLite pour les métadonnées de sessions, HDF5 compressé pour les signaux haute fréquence, et fichiers plats (CSV + MP4) pour les timestamps vidéo.

Le système est doté d'une interface graphique développée avec PyQt6, accessible via un login administrateur unique, permettant la configuration des sessions, la visualisation temps réel des signaux, et la consultation du dataset constitué. Les sujets sont identifiés par un identifiant séquentiel anonyme généré automatiquement (`subject_001`, `subject_002`...). Les seules métadonnées collectées sont génériques et non nominatives : tranche d'âge, genre, port de lunettes et conditions médicales éventuelles. Les données sont organisées selon quatre classes comportementales inspirées du dataset DMD de Vicomtech : distraction, fatigue, attentiveness et gaze.

Mots-clés : Driver Monitoring System, EEG, Synchronisation multisensorielle, HDF5, SQLite, PyQt6, Dataset, Fatigue au volant, NeuroSky MindWave, ORBEC Astra Pro, Logitech G29.

Abstract

Driver monitoring represents a critical road safety challenge, particularly for professional drivers exposed to fatigue and distraction over long distances. While advanced driver assistance systems (ADAS) exist in high-end vehicles, they remain inaccessible for the vast majority of heavy trucks operating in Morocco, due to the lack of affordable and locally adapted solutions.

This final year project proposes the design and development of a multimodal driver monitoring prototype, named DMS DATA COLLECTOR. The main objective is to build a data collection, synchronization, and structuring pipeline for heterogeneous signals from four devices : an EEG headset (NeuroSky MindWave Mobile 2), a biometric smartwatch (iSo Tech), a 3D camera (ORBEC Astra Pro), and a driving simulator (Logitech G29 Driving Force).

Temporal synchronization is ensured by a shared software clock mechanism (SyncClock), based on Python's `time.perf_counter()`, guaranteeing sub-millisecond precision without external dependencies. Storage uses a hybrid three-layer architecture : SQLite for session metadata, compressed HDF5 for high-frequency signals, and flat files (CSV + MP4) for video timestamps. Subjects are identified by an automatically generated anonymous sequential ID, with only generic metadata collected (age range, gender, glasses, medical conditions).

Keywords : Driver Monitoring System, EEG, Multisensory Synchronization, HDF5, SQLite, PyQt6, Dataset, Driving Fatigue, NeuroSky MindWave, ORBEC Astra Pro, Logitech G29.

Remerciements

Avant tout, nous tenons à exprimer notre profonde gratitude au Prof. Asmaa El HANNANI, notre encadrante de projet, pour sa disponibilité constante, ses conseils avisés et son encadrement rigoureux tout au long de ce projet. Son expertise, sa patience et ses encouragements nous ont permis de structurer notre démarche avec rigueur et de surmonter les obstacles techniques rencontrés.

Nous adressons également nos sincères remerciements à Madame Ghita LAYOUNE, doctorante accompagnatrice du projet, pour son soutien technique, ses suggestions pertinentes concernant la structure du dataset et les métadonnées à collecter, ainsi que pour sa réactivité lors de nos échanges. Ses conseils ont enrichi notre approche et contribué à l'amélioration de notre travail.

Nous remercions vivement les membres du jury pour le temps consacré à l'évaluation de notre travail et pour leurs remarques constructives qui enrichiront notre réflexion.

Nos remerciements vont également à l'ensemble du corps professoral de l'École Nationale des Sciences Appliquées d'El Jadida, pour la qualité de leur enseignement et leur engagement dans la formation des futurs ingénieurs.

Enfin, nous remercions nos familles, nos amis et toutes les personnes qui nous ont soutenues, de près ou de loin, dans la concrétisation de ce projet de fin d'année.

Concernant les travaux antérieurs et le contexte de ce projet, plusieurs éléments avaient déjà été identifiés avant notre intervention, notamment l'étude préliminaire des capteurs disponibles, la définition des besoins fonctionnels de base et l'identification du besoin de synchronisation multisensorielle. Notre travail s'inscrit ainsi dans la continuité de ces réflexions initiales.

Notre contribution personnelle dans ce projet se manifeste à travers le développement du prototype DMS Data Collector en Python, incluant l'implémentation du module de synchronisation SyncClock basé sur `time.perf_counter()`, la conception du pipeline de collecte multi-threadé pour les différents capteurs, la structuration du dataset au format HDF5 avec compression, le reverse-engineering du protocole BLE du bracelet iSo Tech, ainsi que le développement de l'interface graphique PyQt6.

Nous avons également réalisé des sessions de test sur simulateur de conduite, validé la synchronisation inter-capteurs avec une précision élevée et constitué un premier dataset structuré selon plusieurs classes comportementales. L'ensemble du code et des protocoles développés a été documenté afin de garantir la reproductibilité et l'évolutivité du système.

Table des matières

Résumé	i
Abstract	ii
Remerciements	iii
Liste des figures	vii
Liste des tableaux	viii
Liste des abréviations	ix
Introduction générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du Projet	2
1.2.1 Problématique	2
1.2.2 Étude de l'existant	3
1.2.3 Critique de l'existant	4
1.2.4 Solution proposée	4
1.3 Méthodologie de Travail	5
1.3.1 Approche itérative	5
1.3.2 Spécifications du système	6
1.3.3 Planification du projet	7
1.4 Conclusion	8
2 Concepts de base et état de l'art	9
2.1 Introduction	9
2.2 Notions Fondamentales	9
2.2.1 Surveillance du conducteur	9
2.2.2 Signaux physiologiques et comportementaux	9
2.2.3 Synchronisation multisensorielle	10
2.2.4 Distinction entre résolution d'horloge et précision de déclenchement	10

2.2.5	Prise en compte des latences matérielles	11
2.3	Présentation du Matériel	11
2.3.1	Casque EEG — NeuroSky MindWave Mobile 2	13
2.3.2	Bracelet connecté — iSo Tech	13
2.3.3	Caméra 3D — ORBBEC Astra Pro	13
2.3.4	Simulateur de conduite — Logitech G29	14
2.4	Comparaison des Solutions Techniques	15
2.4.1	Solutions de synchronisation	15
2.4.2	Architectures de stockage	15
2.5	Conclusion	16
3	Étude fonctionnelle et conceptuelle	17
3.1	Introduction	17
3.2	Architecture Globale du Système	17
3.3	Diagrammes UML	18
3.3.1	Diagramme de cas d'utilisation	18
3.3.2	Diagramme de classes	19
3.3.3	Diagramme de séquence	20
3.4	Structure du Dataset	21
3.4.1	Classes de conduite	21
3.4.2	Mécanisme d'affectation des classes	22
3.4.3	Validation de la labellisation	22
3.4.4	Organisation et structure des fichiers	23
3.4.5	Anonymat et conformité RGPD	23
3.5	Workflow de l'Application	24
3.6	Conclusion	25
4	Mise en œuvre	26
4.1	Introduction	26
4.2	Environnement de Développement	26
4.3	Module de Synchronisation — SyncClock	27
4.4	Modules d'Acquisition	27
4.4.1	EEGReader — NeuroSky MindWave 2	27
4.4.2	BraceletReader — iSo Tech (H59MAX)	27
4.4.3	CameraReader — ORBBEC Astra Pro	28
4.4.4	WheelReader — Logitech G29	28
4.5	Interface Graphique PyQt6	29
4.5.1	Écran AdminLoginScreen	29
4.5.2	Écran SetupScreen	29
4.5.3	Écran DashboardScreen	30

4.5.4	Écran DatasetScreen	30
4.6	Validation et Tests	31
4.6.1	Interface de Collecte — Tableau de Bord en Temps Réel	32
4.6.2	Trace Console de l'Acquisition EEG	33
4.6.3	Analyse des Signaux EEG Collectés	34
4.6.4	Analyse des Signaux Cardiovasculaires	35
4.6.5	Récapitulatif des Résultats de Validation	37
4.6.6	Fichiers Produits par Session	37
4.7	Conclusion	38
Conclusion générale		39
Bibliographie		41
A Codes Sources Principaux		42
A.1	Module SyncClock	42
A.2	Module WheelReader — Logitech G29	43
A.3	Module CameraReader — Extraction depth	43
A.4	Requête SQL de filtrage du dataset	43
A.5	Alignement inter-capteurs avec pandas	44
B Protocole BLE du Bracelet iSo Tech (H59MAX)		45
C Résultats de Validation — Session de Référence		46
D Structure Complète du Projet		47

Table des figures

Figure 1.1	Diagramme de Gantt — planification du projet DMS Data Collector	8
Figure 2.1	Ensemble du matériel utilisé dans le prototype DMS Data Collector	12
Figure 3.1	Architecture globale du système DMS Data Collector	18
Figure 3.2	Diagramme de cas d'utilisation UML	19
Figure 3.3	Diagramme de classes UML	20
Figure 3.4	Diagramme de séquence — déroulement d'une session complète .	21
Figure 3.5	Arborescence du dataset DMS — structure générée automatiquement	23
Figure 3.6	Workflow de navigation de l'application DMS Data Collector . .	25
Figure 4.1	Architecture à deux threads du module CameraReader	28
Figure 4.2	Écran AdminLoginScreen — authentification administrateur . .	29
Figure 4.3	Écran SetupScreen — configuration complète de la session . . .	29
Figure 4.4	Écran DashboardScreen — visualisation temps réel des signaux	30
Figure 4.5	Écran DatasetScreen — consultation du dataset depuis SQLite .	30
Figure 4.6	Session de collecte réelle : sujet équipé des 4 capteurs	31
Figure 4.7	Tableau de bord DMS Data Collector lors d'une session de collecte (subject_036, classe DISTRACTION)	32
Figure 4.8	Logs console du module EEGReader — valeurs eSense (Attention, Méditation)	33
Figure 4.9	Visualisation des signaux EEG collectés durant une session . . .	34
Figure 4.10	Visualisation des signaux physiologiques collectés par le bracelet	36
Figure 4.11	Fichiers générés pour une session — vue explorateur Windows .	38

Liste des tableaux

Tableau 1.1	Comparaison des datasets de référence pour la surveillance du conducteur	3
Tableau 1.2	Analyse des besoins fonctionnels	6
Tableau 1.3	Analyse des besoins non-fonctionnels	7
Tableau 2.1	Bandes de fréquences EEG et états cognitifs associés	10
Tableau 2.2	Vue d'ensemble du matériel utilisé dans le prototype DMS . . .	12
Tableau 2.3	Caractéristiques techniques du NeuroSky MindWave Mobile 2 .	13
Tableau 2.4	Caractéristiques techniques de la caméra ORBBEC Astra Pro . .	14
Tableau 2.5	Caractéristiques techniques du Logitech G29 Driving Force . . .	14
Tableau 2.6	Comparaison des solutions de synchronisation	15
Tableau 2.7	Benchmark comparatif des architectures de stockage	16
Tableau 3.1	Classes de conduite, activités et signaux discriminants	22
Tableau 3.2	Rôle et fonctionnalités des quatre écrans de l'application	24
Tableau 4.1	Environnement de développement et bibliothèques utilisées . . .	26
Tableau 4.2	Résultats de validation des sessions de test	37

Liste des abréviations

Abréviation	Signification
ADAS	Advanced Driver Assistance Systems
API	Application Programming Interface
BLE	Bluetooth Low Energy
BCI	Brain-Computer Interface
CSV	Comma-Separated Values
DMS	Driver Monitoring System
DMD	Driver Monitoring Dataset
EEG	Électroencéphalographie
FC	Fréquence Cardiaque
GUI	Graphical User Interface
HDF5	Hierarchical Data Format version 5
HID	Human Interface Device
HRV	Heart Rate Variability
ISIC	Ingénierie des Systèmes d'Information et de Communication
ML	Machine Learning (Apprentissage Automatique)
MP4	Format vidéo MPEG-4 Part 14
RGPD	Règlement Général sur la Protection des Données
SDK	Software Development Kit
SpO2	Saturation en Oxygène du Sang
SQL	Structured Query Language
TCP	Transmission Control Protocol
UML	Unified Modeling Language
USB	Universal Serial Bus
UTC	Coordinated Universal Time

Introduction générale

La sécurité routière représente aujourd'hui un enjeu majeur à l'échelle mondiale. Selon l'Organisation Mondiale de la Santé, les accidents de la route causent chaque année environ 1,35 million de décès, faisant de ce phénomène l'une des principales causes de mortalité évitable. Parmi les facteurs humains impliqués, la fatigue, la somnolence et la distraction du conducteur jouent un rôle déterminant, étant responsables d'une part significative des accidents graves, notamment sur les axes autoroutiers à longue distance. [7]

Cette problématique concerne particulièrement les conducteurs professionnels, soumis à de longues périodes de conduite, à des horaires irréguliers et à des contraintes de performance élevées. Au Maroc, la situation est d'autant plus préoccupante que l'ancienneté moyenne du parc de poids lourds limite la présence de systèmes embarqués modernes. Les solutions commerciales de surveillance du conducteur demeurent coûteuses et inaccessibles pour une grande majorité des acteurs du transport.[9]

Sur le plan scientifique, la détection automatique de l'état du conducteur repose sur l'analyse combinée de données issues de sources complémentaires : signaux cérébraux, données physiologiques, paramètres de conduite et analyse visuelle du regard. L'exploitation conjointe de ces données nécessite leur collecte synchronisée et leur structuration au sein de datasets annotés, exploitables par des modèles d'apprentissage automatique.

C'est dans ce contexte que s'inscrit ce Projet de Fin d'Année, consacré au développement de DMS DATA COLLECTOR, un prototype logiciel dédié à la collecte et à la gestion de données multisensorielles pour la surveillance du conducteur. Le système acquiert, synchronise en temps réel et stocke des données provenant de quatre dispositifs : un casque EEG, un bracelet biométrique, une caméra 3D et un simulateur de conduite. L'objectif principal est de constituer une infrastructure de collecte fiable et reproductible, première brique d'un futur système de détection automatique des états à risque au volant.

Ce rapport s'articule autour de quatre chapitres. Le premier présente le contexte général, l'étude de l'existant et la méthodologie adoptée. Le deuxième introduit les concepts fondamentaux, le matériel utilisé et les solutions techniques étudiées. Le troisième expose l'architecture du système, les diagrammes UML et la structure du dataset. Le quatrième détaille l'implémentation technique et présente les résultats de validation obtenus.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre vise à cadrer notre projet en exposant la problématique à laquelle nous répondons, les solutions existantes et leurs limites, ainsi que notre approche pour y remédier. Il présente également la méthodologie adoptée et la planification du travail effectué tout au long du semestre.

1.2 Présentation du Projet

1.2.1 Problématique

L'industrie du transport routier fait face à une réalité alarmante : la fatigue, la distraction et la microsommeil constituent les principales causes d'accidents graves impliquant des poids lourds. Des études menées par le Centre Européen de Sécurité Routière estiment que la somnolence est responsable de 10 à 20 % des accidents sur voie rapide, avec un risque particulièrement élevé entre 2h et 6h du matin et entre 14h et 16h. Ces accidents surviennent souvent sans réaction du conducteur, ce qui les rend particulièrement mortels.

Dans ce contexte, les systèmes de surveillance du conducteur (*Driver Monitoring Systems* ou DMS) émergent comme une solution prometteuse. Ces systèmes analysent en temps réel l'état physique et cognitif du conducteur à partir de signaux multimodaux pour détecter les signes précurseurs de défaillance et déclencher des alertes préventives. Cependant, le développement de tels systèmes intelligents repose sur l'existence de datasets de qualité, suffisamment riches et diversifiés pour entraîner et valider des modèles de classification fiables.

Problématique centrale

Comment concevoir un prototype multisensoriel capable de collecter, synchroniser et structurer des données hétérogènes issues des différents types de capteurs, de manière à constituer un dataset labellisé, anonymisé et conforme aux standards de la recherche en apprentissage automatique, dans un contexte de laboratoire à ressources limitées ?

1.2.2 Étude de l'existant

Plusieurs datasets de référence ont été développés par la communauté scientifique pour la reconnaissance des états du conducteur ou des états cognitifs. Le Tableau 1.1 présente une comparaison des datasets les plus significatifs, en mettant en évidence les modalités collectées, le nombre de sujets, la méthode de synchronisation et les conditions d'accès.

TABLEAU 1.1 – Comparaison des datasets de référence pour la surveillance du conducteur

Dataset	Signaux collectés	Sujets	Synchro.	Accès
MAHNOB-HCI (2011)	EEG, ECG, Respiration, Gaze, Vidéo	27	Horloge PC	Académique
DEAP (2012)	EEG 32 canaux, GSR, Pléthysmo., Vidéo	32	elapsed_s commun	Académique
SEED-IV (2018)	EEG 62 canaux, Signaux oculaires	15	Horloge mono-PC	Académique
DMD (2020)	Vidéo RGB + IR, Comportement	37	Capture vidéo sync.	Académique

Ces datasets constituent une base importante pour la recherche, mais présentent des différences significatives en termes de modalités collectées et de méthodes de synchronisation. L'analyse comparative révèle trois constats majeurs pour notre projet :

Premièrement, aucun dataset existant ne combine simultanément des signaux EEG, des données physiologiques cardiovasculaires et des paramètres de conduite active issus d'un simulateur. Le dataset MAHNOB-HCI intègre l'EEG et l'ECG mais ne capture pas les interactions avec un volant. Le dataset DMD se concentre exclusivement sur la modalité vidéo et comportementale, sans données physiologiques.

Deuxièmement, les méthodes de synchronisation employées varient considérablement. MAHNOB-HCI et SEED-IV utilisent une horloge PC centralisée similaire à notre approche SyncClock, tandis que DMD repose sur la capture vidéo synchronisée. Notre solution s'aligne sur les standards académiques établis par MAHNOB-HCI, tout en les adaptant à un contexte matériel accessible.

Troisièmement, le nombre de sujets dans les datasets existants (15 à 37) fournit une référence pour dimensionner notre future campagne de collecte. Notre architecture permet un nombre variable de sujets, offrant une flexibilité adaptée aux contraintes de notre environnement de laboratoire.

Ces observations confirment la pertinence de développer un dataset multimodal combinant EEG, signaux physiologiques et données de conduite — une combinaison actuellement absente du paysage scientifique, et qui constitue l’une des originalités de notre approche. [1][3][4]

1.2.3 Critique de l’existant

L’analyse des solutions existantes révèle plusieurs limitations qui justifient le développement d’un nouveau prototype.

Limitation économique : Les datasets de référence sont construits dans des contextes où le coût des équipements est rarement une contrainte. Le dataset MAHNOB-HCI utilise un amplificateur EEG Biosemi ActiveTwo à 64 canaux dont le coût dépasse 20 000 €. Le dataset SEED-IV mobilise un système ESI NeuroScan à 62 canaux d’une valeur équivalente. Ces investissements sont incompatibles avec les budgets de recherche limités, particulièrement dans les économies émergentes.

Limitation de reproductibilité : Aucune des solutions existantes ne publie de pipeline open-source complet permettant la reproduction intégrale du protocole de collecte. Les articles associés décrivent les méthodologies mais ne fournissent pas le code source des modules d’acquisition, de synchronisation et de stockage. Cette absence constitue un frein majeur à la démocratisation de la recherche en surveillance du conducteur.

Limitation technique de synchronisation : Les méthodes de synchronisation utilisées sont soit basées sur des triggers hardware difficilement reproductibles (MAHNOB-HCI avec triggers parallèles), soit sur des middleware réseau comme Lab Streaming Layer (LSL), surdimensionnés pour une architecture mono-machine. LSL introduit une complexité d’installation et une latence réseau non nécessaires lorsque tous les capteurs sont connectés à un unique poste de travail.

Limitation de couverture modale : Le dataset DMD de Vicomtech, bien que constituant une référence pour l’analyse comportementale, se limite aux modalités vidéo RGB et infrarouge. Il ne capture ni l’activité cérébrale (EEG) ni les signaux physiologiques cardiovasculaires, qui sont pourtant des indicateurs précoces de la fatigue et de la somnolence.

Ces quatre limitations — économique, de reproductibilité, technique et de couverture modale — définissent précisément l’espace d’innovation dans lequel s’inscrit notre projet DMS DATA COLLECTOR.

1.2.4 Solution proposée

Pour répondre à ces limitations, notre projet propose DMS DATA COLLECTOR : une solution logicielle low-cost et reproductible, basée sur une horloge logicielle partagée (SyncClock)

à précision sub-milliseconde, une architecture de stockage hybride adaptée aux différents types de données, et un respect structurel de l'anonymat des sujets par génération automatique d'identifiants séquentiels.

Plus spécifiquement, notre solution apporte les contributions suivantes :

1. **Accessibilité économique** : Le coût total du matériel est inférieur à 500 €, rendant le système déployable dans des contextes budgétaires contraints, y compris pour les flottes de transport dans les pays en développement.
2. **Reproductibilité totale** : L'intégralité du code source est structurée en modules Python indépendants et documentés, permettant à toute équipe de recherche de reproduire le pipeline de collecte sur son propre matériel.
3. **Synchronisation simplifiée** : Le mécanisme `SyncClock`, basé sur `time.perf_counter()`, fonctionne sans dépendance externe ni configuration réseau, avec une précision inférieure à 1 ms validée expérimentalement.
4. **Couverture multimodale inédite** : Notre système est le premier à combiner simultanément l'EEG frontal, les signaux cardiovasculaires (FC, SpO2, HRV), les paramètres de conduite (direction, accélération, freinage) et la vidéo 3D (RGB + profondeur) au sein d'un dataset unifié et synchronisé.

1.3 Méthodologie de Travail

1.3.1 Approche itérative

Le développement du système a été réalisé selon une approche itérative et incrémentale, inspirée des méthodologies agiles. Ce choix s'est imposé naturellement compte tenu de la nature exploratoire du projet, qui repose sur l'intégration de matériels hétérogènes et l'émergence progressive des exigences fonctionnelles au fil des tests et des retours obtenus lors des réunions de suivi.

Chaque itération comprenait quatre phases successives : l'analyse des exigences issues de la réunion précédente, la conception et l'implémentation des fonctionnalités planifiées, les tests de validation sur machine de développement, et la préparation du livrable de réunion. Cette approche a permis de faire évoluer l'application depuis la première version — simple collecteur CSV — jusqu'à la version finale dotée d'une architecture SQLite/HDF5 et d'une interface administrateur complète.

1.3.2 Spécifications du système

Les spécifications du système ont été élaborées en distinguant les exigences fonctionnelles et non fonctionnelles. Le Tableau 1.2 résume les principales exigences fonctionnelles identifiées au cours de l'analyse des besoins.

TABLEAU 1.2 – Analyse des besoins fonctionnels

ID	Besoin fonctionnel	Description	Priorité
BF-01	Authentification	Connexion admin par mot de passe fixe	Haute
BF-02	Configuration	Sujet, classe, activité, durée, modes capteurs	Haute
BF-03	Collecte multimodale	Acquisition simultanée des 4 capteurs	Haute
BF-04	Synchronisation	Timestamp <code>elapsed_s</code> commun via <code>SyncClock</code>	Haute
BF-05	Mode simulation	Basculement automatique si capteur absent	Haute
BF-06	Visualisation temps réel	8 panneaux mini-graphiques dynamiques	Moyenne
BF-07	Annotation	Colonne <code>label</code> dans chaque fichier	Haute
BF-08	Validation post-session	Vérification cohérence des données produites	Haute
BF-09	Organisation dataset	Structuration hiérarchique classe/sujet/session	Haute
BF-10	Anonymat	Génération automatique du <code>subject_id</code>	Haute

Le Tableau 1.3 présente quant à lui les exigences non fonctionnelles, qui fixent les critères de qualité auxquels le système doit satisfaire indépendamment des fonctionnalités offertes.

TABLEAU 1.3 – Analyse des besoins non-fonctionnels

ID	Besoin	Critère de satisfaction
BNF-01	Précision temporelle	Erreur d'alignement inter-capteurs < 50 ms
BNF-02	Performance temps réel	Écriture continue sans perte à 30 Hz
BNF-03	Conformité RGPD	Aucune donnée nominative dans le dataset
BNF-04	Robustesse	Basculement simulation automatique en cas de défaillance
BNF-05	Portabilité	Windows 10/11, sans infrastructure réseau spéciale
BNF-06	Extensibilité	Architecture modulaire permettant l'ajout de capteurs
BNF-07	Interopérabilité	Fichiers lisibles par pandas, numpy, scikit-learn

1.3.3 Planification du projet

La planification du projet a été répartie sur douze semaines, organisées en phases successives allant de l'analyse bibliographique jusqu'à la préparation de la soutenance. La Figure 1.1 illustre le diagramme de Gantt correspondant à cette organisation.

Phase 1 — Analyse et conception (semaines 1-2)

Phase 2 — Développement modulaire (semaines 3-6)

Phase 3 — Intégration et interface (semaines 7-9)

Phase 4 — Tests et validation (semaines 8-10)

Phase 5 — Rédaction et finalisation (semaines 9-12)

Le chevauchement volontaire entre les phases de développement, de test et de rédaction reflète l'approche itérative adoptée. Cette stratégie a permis d'ajuster continuellement les choix techniques en fonction des résultats expérimentaux, garantissant la cohérence entre les objectifs initiaux et les réalisations finales.

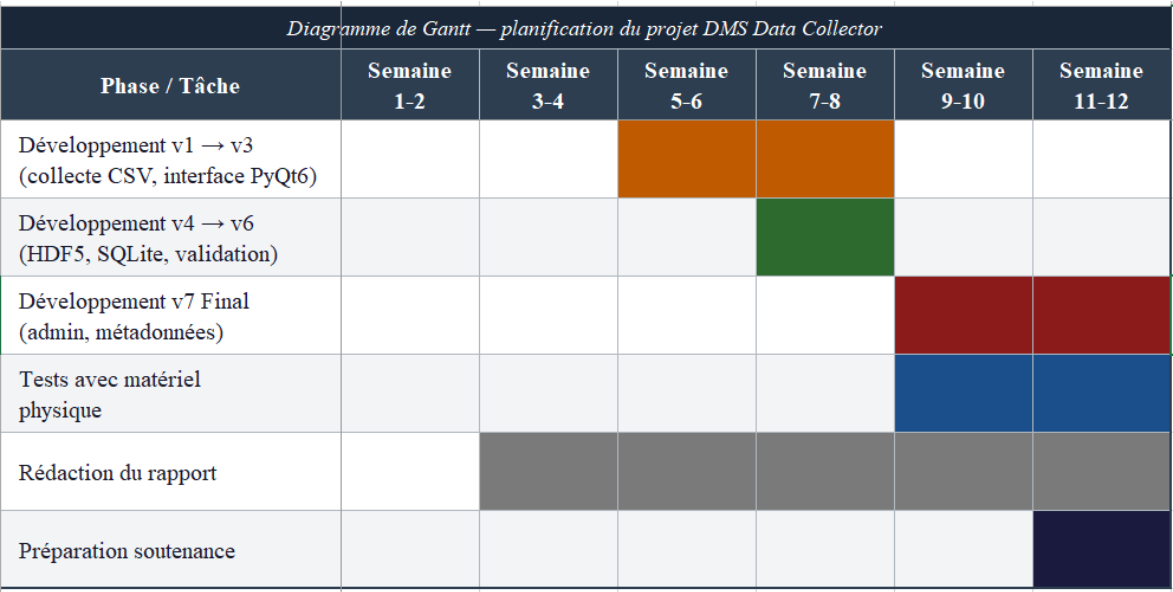


FIGURE 1.1 – Diagramme de Gantt — planification du projet DMS Data Collector

Le diagramme de Gantt met en évidence le chevauchement volontaire entre les phases de développement et de rédaction du rapport, conformément à l’approche itérative adoptée. Les tests avec le matériel physique ont été conduits en parallèle du développement de la version finale, ce qui a permis d’ajuster les paramètres d’acquisition au fur et à mesure des résultats obtenus.

1.4 Conclusion

Ce chapitre a permis de poser le cadre général de notre projet en définissant la problématique centrale, en analysant les solutions existantes et leurs limites, et en présentant la solution proposée. La méthodologie itérative adoptée et la planification rigoureuse du travail constituent les fondements sur lesquels repose l’ensemble du développement présenté dans les chapitres suivants.

Chapitre 2

Concepts de base et état de l'art

2.1 Introduction

Ce chapitre introduit les notions fondamentales nécessaires à la compréhension du projet. Il couvre les concepts liés à la surveillance du conducteur, les signaux physiologiques et comportementaux exploités, ainsi que la problématique de synchronisation multisensorielle. Une présentation du matériel utilisé est ensuite proposée, suivie d'une comparaison des solutions techniques étudiées pour la synchronisation et le stockage des données.

2.2 Notions Fondamentales

2.2.1 Surveillance du conducteur

La surveillance du conducteur (*Driver Monitoring System*, DMS) constitue un domaine de recherche à l'intersection de la sécurité routière, du traitement du signal et de l'intelligence artificielle. Elle vise à évaluer en temps réel l'état cognitif et comportemental du conducteur afin de détecter et d'anticiper les situations à risque. Selon l'OMS, la distraction au volant est impliquée dans environ 25 % des accidents de la route dans le monde, et la somnolence est responsable d'une proportion significative des accidents mortels sur autoroute. On distingue trois catégories d'états surveillés : la distraction, caractérisée par un détournement de l'attention vers une activité secondaire ; la fatigue et la somnolence, qui se manifestent par des modifications physiologiques mesurables ; et l'inattention au regard, définie par l'orientation prolongée des yeux vers des zones non pertinentes pour la conduite.

2.2.2 Signaux physiologiques et comportementaux

L'électroencéphalographie (EEG) mesure l'activité électrique du cerveau à travers des électrodes placées sur le cuir chevelu. Le signal est décomposé en bandes de fréquences dont les variations permettent d'inférer l'état cognitif du sujet. Le Tableau 2.1 résume les cinq bandes principales et les états cognitifs qui leur sont associés.

TABLEAU 2.1 – Bandes de fréquences EEG et états cognitifs associés

Bande	Fréquence	Amplitude	État cognitif associé
Delta (δ)	0,5 – 4 Hz	20 – 200 μ V	Sommeil profond, somnolence sévère
Thêta (θ)	4 – 8 Hz	5 – 100 μ V	Somnolence, fatigue cognitive
Alpha (α)	8 – 13 Hz	2 – 60 μ V	Relaxation éveillée, yeux fermés
Bêta (β)	13 – 30 Hz	1 – 30 μ V	Attention active, concentration
Gamma (γ)	30 – 100 Hz	1 – 5 μ V	Traitement cognitif avancé

En complément de l'EEG, trois indicateurs physiologiques cardiovasculaires sont collectés via le bracelet biométrique. La fréquence cardiaque (FC) tend à diminuer progressivement lors d'une fatigue de conduite prolongée. La saturation en oxygène (SpO2) est normalement comprise entre 95 et 99 % et peut baisser en cas de somnolence avancée. La variabilité de la fréquence cardiaque (HRV) est particulièrement discriminante : une HRV élevée est associée à un état d'éveil, tandis qu'une HRV faible et régulière caractérise un état de stress ou de fatigue avancée. Les données de conduite, capturées via le simulateur à 30 Hz, fournissent l'angle de braquage, la pression sur l'accélérateur et la fréquence de freinage — des signatures comportementales directement liées à l'état d'attention du conducteur.

2.2.3 Synchronisation multisensorielle

Les quatre capteurs du prototype présentent des caractéristiques temporelles très différentes : le casque EEG et le bracelet produisent des données à 1 Hz via des protocoles sans fil (Bluetooth SPP et BLE) avec des latences variables, le simulateur fonctionne à 30 Hz via USB HID avec une latence inférieure à 1 ms, et la caméra capture à 20 fps via USB OpenNI2 avec un délai de l'ordre de 5 à 10 ms. Cette hétérogénéité rend impossible l'utilisation d'une horloge matérielle commune et impose une solution logicielle de synchronisation adaptée.

2.2.4 Distinction entre résolution d'horloge et précision de déclenchement

Dans le contexte de la collecte de données multisensorielles, il est essentiel de distinguer deux notions complémentaires mais souvent confondues. La résolution d'horloge représente le plus

petit intervalle de temps mesurable par le système. Sur Windows, `time.perf_counter()` offre une résolution inférieure à 100 nanosecondes, ce qui signifie que deux événements espacés de 100 ns peuvent théoriquement être distingués. En revanche, la précision de déclenchement correspond à la capacité à déclencher plusieurs capteurs au même instant théorique. Par exemple, lorsqu'on souhaite que le casque EEG et le bracelet acquièrent une donnée précisément à $t = 1$ seconde, la précision mesure l'écart réel entre leurs instants d'acquisition. Un capteur avec une horloge précise mais un déclenchement imprécis pourrait acquérir à $t = 1,050$ s, et l'écart de 50 ms entre les deux capteurs constitue l'erreur de synchronisation.

2.2.5 Prise en compte des latences matérielles

Chaque composant matériel introduit des latences qu'il faut impérativement prendre en considération dans le calcul du timestamp final. Le casque EEG NeuroSky présente une latence estimée entre 10 et 30 ms due au traitement Bluetooth et au filtrage du signal. Le bracelet iSo Tech, communiquant en BLE, introduit une latence plus importante de l'ordre de 50 à 200 ms, ce qui impacte directement l'alignement temporel des mesures de fréquence cardiaque et de saturation en oxygène. Le volant Logitech G29, via son interface USB HID, affiche une latence inférieure à 1 ms, ce qui est négligeable pour notre application. Enfin, la caméra ORBBEC Astra Pro, utilisant OpenNI2 sur USB, présente une latence de 5 à 10 ms pour l'acquisition des frames RGB et de profondeur. Pour résoudre ce problème de synchronisation, notre système DMS Data Collector implémente une horloge de référence unique SyncClock basée sur `time.perf_counter()`, un timestamping à la source pour chaque échantillon via `SyncClock.elapsed()`, et une vérification post-collecte de la cohérence temporelle via `pandas.merge_asof()`. L'alignement inter-capteurs est ainsi réalisé avec une marge d'erreur inférieure à 50 ms, ce qui est jugé acceptable car pour l'EEG et le bracelet à 1 Hz, 50 ms représente moins de 5% de la fenêtre d'échantillonnage de 1000 ms.

2.3 Présentation du Matériel

Le Tableau 2.2 donne une vue d'ensemble des quatre dispositifs utilisés dans le prototype. La Figure 2.1 illustre l'ensemble du matériel tel qu'il a été utilisé lors des sessions de collecte.

TABLEAU 2.2 – Vue d’ensemble du matériel utilisé dans le prototype DMS

Équipement	Signal collecté	Protocole	Fréquence	Coût approx.
NeuroSky Mind-Wave 2	EEG + Attention/Méditation	BT ThinkGear TCP	1 Hz	~100 €
Bracelet iSo Tech	FC, SpO2, HRV, Stress	BLE 4.0	1 Hz	~50 €
ORBEC Astra Pro	RGB + Profondeur	USB OpenNI2	20/30 fps	~200 €
Logitech G29	Steering, Throttle, Brake	USB HID	30 Hz	~150 €

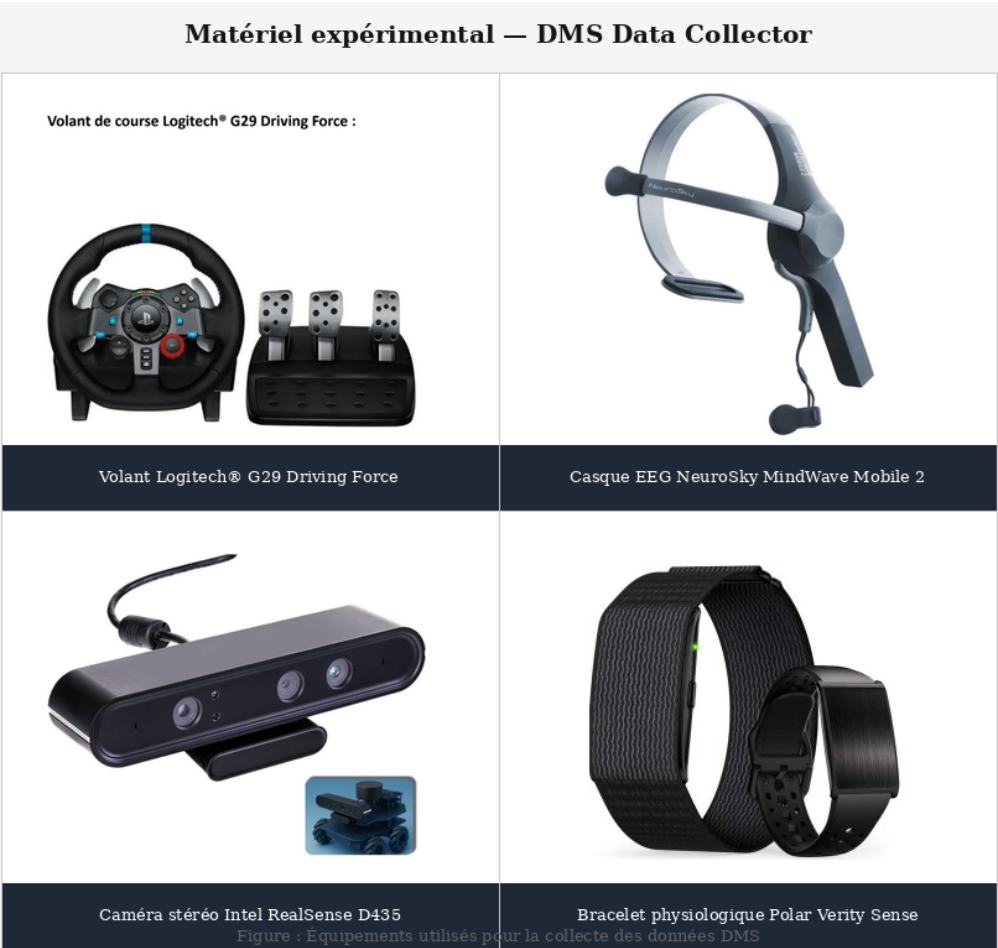


FIGURE 2.1 – Ensemble du matériel utilisé dans le prototype DMS Data Collector

Comme le montre la Figure 2.1, les quatre dispositifs sont connectés simultanément à un seul poste de travail Windows via leurs interfaces respectives. Cette architecture mono-machine simplifie la synchronisation et évite les problèmes de dérive d’horloge entre plusieurs ordinateurs.

2.3.1 Casque EEG — NeuroSky MindWave Mobile 2

Le NeuroSky MindWave Mobile 2 est un casque EEG grand public composé d'une électrode frontale unique (FP1) et d'un clip auriculaire servant de référence. Le module TGAM1 embarqué effectue le traitement du signal en temps réel et transmet les données via Bluetooth SPP vers le ThinkGear Connector, un service local accessible sur le port TCP 13854. Les signaux fournis incluent les spectres de puissance décomposés en huit bandes fréquentielles (delta, thêta, alpha bas/haut, bêta bas/haut, gamma bas/milieu) ainsi que deux indicateurs eSense — l'attention et la méditation — normalisés entre 0 et 100.[5] Le Tableau 2.3 résume les caractéristiques techniques de ce dispositif.

TABLEAU 2.3 – Caractéristiques techniques du NeuroSky MindWave Mobile 2

Paramètre	Valeur
Protocole de communication	Bluetooth SPP (socket TCP port 13854)
Fréquence d'échantillonnage	512 Hz (Raw EEG) — 1 Hz (eSense)
Résolution ADC	12 bits
Autonomie	8 heures (1 pile AAA)
Électrode active	FP1 (frontale gauche)

2.3.2 Bracelet connecté — iSo Tech

Le bracelet iSo Tech est un capteur biométrique portable communiquant en BLE 4.0 permettant la mesure continue de la fréquence cardiaque, de la saturation en oxygène (SpO2), de la variabilité de la fréquence cardiaque (HRV) et d'un indicateur de stress. Son protocole de communication est propriétaire : la commande de démarrage du stream de mesures vitals a été identifiée par analyse des trames BLE capturées lors des sessions de test. La connexion s'effectue par adresse MAC via la bibliothèque Python Bleak, avec une notification de données à une fréquence approximative d'une trame toutes les trois secondes. Le détail complet de la structure des trames est disponible en Annexe B.

2.3.3 Caméra 3D — ORBBEC Astra Pro

La caméra ORBBEC Astra Pro est une caméra de profondeur RGB-D utilisant la technologie de lumière structurée monoculaire. Elle capture simultanément un flux RGB et une carte de profondeur, toutes deux accessibles via le SDK OpenNI2.[6] Le Tableau 2.4 présente ses caractéristiques techniques. Dans notre implémentation, le flux RGB est capturé via OpenCV

à 20 fps et le flux de profondeur via OpenNI2 à 29,3 fps réels mesurés, dans une résolution de 640×480 pixels et avec une plage de mesure validée entre 443 mm et 1925 mm lors des sessions de test.

TABLEAU 2.4 – Caractéristiques techniques de la caméra ORBBEC Astra Pro

Paramètre	Valeur
Technologie de profondeur	Lumière structurée monoculaire
Plage de fonctionnement	0,6 – 8 m
Précision	±3 mm à 1 m
Résolution RGB	1920×1080 @ 30 fps (max)
Résolution profondeur	640×480 @ 30 fps
Interface	USB 2.0
SDK compatibles	OpenNI2, ROS, OpenCV

2.3.4 Simulateur de conduite — Logitech G29

Le Logitech G29 Driving Force est un volant de simulation à 900° de rotation équipé d'un capteur de direction à effet Hall garantissant une haute précision de mesure. Il est reconnu nativement en USB HID par Windows sans installation de pilote supplémentaire, et accessible via la bibliothèque pygame. Les axes fournis couvrent la direction (steering, $[-1; +1]$), l'accélérateur (throttle), le frein (brake) et l'embrayage (clutch), tous normalisés en $[0; 1]$ après inversion de l'axe brut. Le Tableau 2.5 détaille ses caractéristiques principales.

TABLEAU 2.5 – Caractéristiques techniques du Logitech G29 Driving Force

Paramètre	Valeur
Rotation	900° de butée à butée
Capteur de direction	Effet Hall
Interface PC	USB HID (plug and play)
Axes collectés	steering, throttle, brake, clutch
Fréquence d'acquisition	30 Hz (via pygame)

2.4 Comparaison des Solutions Techniques

2.4.1 Solutions de synchronisation

Trois approches ont été étudiées pour la synchronisation temporelle des capteurs. Le Tableau 2.6 en présente la comparaison selon les critères les plus pertinents pour notre contexte.

TABLEAU 2.6 – Comparaison des solutions de synchronisation

Critère	SyncClock PC	LSL	Horloge capteur
Résolution temporelle	< 100 ns	~1 ms	Non disponible
Infrastructure requise	Aucune	Installation LSL	SDK propriétaire
Compatible BLE	Oui	Non	Non
Compatible pygame/USB	Oui	Non	Non
Dérive inter-capteurs	Nulle	Faible	Non corrigéable
Architecture mono-machine	Optimal	Surdimensionné	Inapplicable
Référence dans la littérature	MAHNOB-HCI, DEAP	—	—

L'approche SyncClock basée sur `time.perf_counter()` a été retenue car elle est la seule à satisfaire simultanément l'ensemble des contraintes du projet : compatibilité avec les quatre protocoles hétérogènes, architecture mono-machine, précision sub-milliseconde et absence de dépendance externe. Cette approche est également celle utilisée par les datasets de référence MAHNOB-HCI et DEAP, ce qui garantit une conformité méthodologique avec l'état de l'art.

2.4.2 Architectures de stockage

Le stockage des données DMS pose une difficulté particulière en raison de l'hétérogénéité des types de données : signaux haute fréquence, flux vidéo, métadonnées structurées. Le Tableau 2.7 compare les quatre principales solutions étudiées selon des critères techniques et pratiques.

TABLEAU 2.7 – Benchmark comparatif des architectures de stockage

Critère	SQLite	HDF5	MongoDB	CSV/MP4
Signaux haute fréquence (30 Hz)	Insuffisant	Excellent	Moyen	Possible
Flux vidéo MP4	Impossible	Partiel	Impossible	Natif
Métadonnées et requêtes SQL	Excellent	Aucune	Bon	Aucune
Compression intégrée	Partielle	gzip	Partielle	Non
Compatibilité pandas/-numpy	Oui	Natif	Via connecteur	Oui
Installation	Aucune	pip h5py	Serveur requis	Aucune

Ce benchmark justifie le recours à une architecture hybride à trois couches complémentaires : SQLite pour les métadonnées de sessions et les requêtes de filtrage du dataset, HDF5 compressé pour les signaux physiologiques et comportementaux haute fréquence, et fichiers plats (CSV + MP4) pour les timestamps vidéo et le flux RGB de la caméra.

2.5 Conclusion

Ce chapitre a présenté les fondements théoriques et techniques du projet, depuis les concepts de surveillance du conducteur jusqu'aux choix technologiques retenus pour la synchronisation et le stockage. Les comparaisons effectuées justifient l'architecture finale du système, qui sera décrite en détail dans le chapitre suivant.

Chapitre 3

Étude fonctionnelle et conceptuelle

3.1 Introduction

Ce chapitre présente l'analyse approfondie des besoins du système, son architecture globale, la modélisation UML et la structure du dataset constitué. Il constitue le pont entre les concepts théoriques du chapitre précédent et l'implémentation technique décrite au chapitre suivant.

3.2 Architecture Globale du Système

Le système DMS DATA COLLECTOR s'organise en cinq couches fonctionnelles distinctes. La couche matérielle regroupe les quatre capteurs physiques connectés au PC via leurs protocoles respectifs. La couche d'acquisition comprend quatre threads Python indépendants (EEGReader, BraceletReader, CameraReader, WheelReader), chacun responsable d'un capteur. La couche de synchronisation est assurée par le module SyncClock, partagé par tous les threads via une variable de classe. La couche de stockage orchestrée par SessionStorage gère les trois formats hybrides (SQLite, HDF5, fichiers plats). Enfin, la couche interface expose quatre écrans PyQt6 au sein de la MainWindow.[10]

Cette architecture en couches répond à un principe fondamental de génie logiciel : la séparation des préoccupations (separation of concerns). Chaque couche est responsable d'un aspect distinct du système et communique avec les couches adjacentes via des interfaces explicites. La couche matérielle isole les spécificités des protocoles de communication (Bluetooth SPP, BLE, USB HID, USB OpenNI2). La couche d'acquisition encapsule les algorithmes de parsing propres à chaque capteur. La couche de synchronisation fournit un service transversal de référence temporelle. La couche de stockage abstrait la complexité du stockage hybride SQLite/HDF5/CSV. Enfin, la couche interface expose les données à l'utilisateur sans exposer les détails d'implémentation sous-jacents.

Cette modularité est essentielle pour la maintenabilité à long terme du système : l'ajout d'un nouveau capteur ne nécessite que la création d'un nouveau reader et son enregistrement dans la configuration, sans modification des couches supérieures. De même, le remplacement de PyQt6 par un autre framework GUI n'impacterait que la couche interface, les autres couches

restant inchangées.

La Figure 3.1 illustre l’architecture globale du système et les interactions entre ces cinq couches.

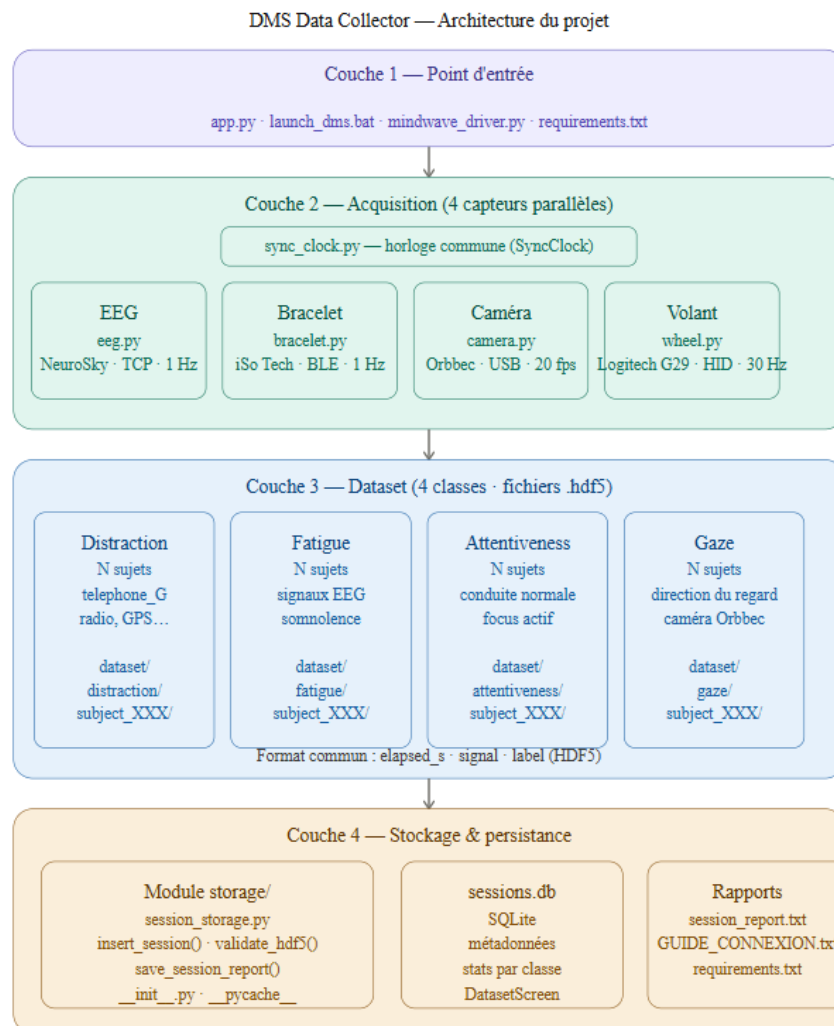


FIGURE 3.1 – Architecture globale du système DMS Data Collector

3.3 Diagrammes UML

3.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Figure 3.2) représente les interactions entre l'administrateur — seul utilisateur de l'application — et les fonctionnalités offertes par le système. Suite à la décision prise lors de la réunion N°4, l'inscription et la connexion des sujets ont été supprimées au profit d'un login administrateur unique, simplifiant considérablement le flux d'utilisation.

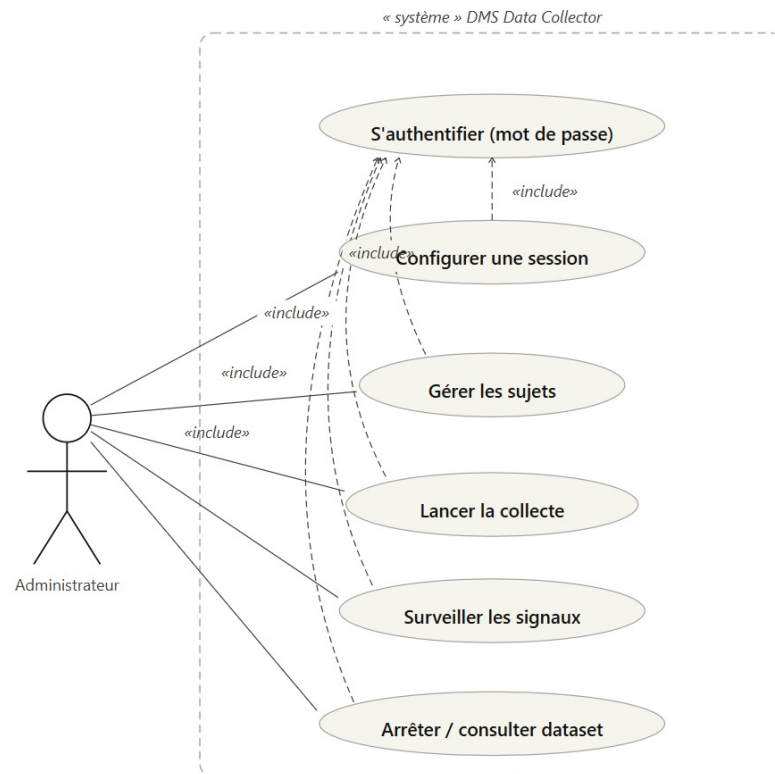


FIGURE 3.2 – Diagramme de cas d'utilisation UML

3.3.2 Diagramme de classes

Le diagramme de classes (Figure 3.3) présente les principales classes du système et leurs relations. La classe `SyncClock` utilise le pattern Singleton via des `@classmethod`, garantissant qu'un seul instant `T0` est partagé par tous les threads. Les quatre readers héritent d'une interface commune et communiquent avec la `MainWindow` uniquement via l'attribut `self.last`, évitant tout couplage direct entre threads.

La conception orientée objet du système repose sur plusieurs principes fondamentaux. L'héritage entre `BaseReader` et les quatre readers concrets garantit un comportement polymorphique : la `MainWindow` peut démarrer, arrêter et interroger n'importe quel capteur via l'interface abstraite, sans connaître son implémentation spécifique. L'encapsulation des données dans chaque reader protège les buffers d'acquisition contre les accès concurrents, chaque thread étant le seul propriétaire de ses données. Le couplage faible entre les readers est assuré par l'absence de référence directe entre eux : ils partagent uniquement l'accès au `SyncClock` via sa variable de classe.

Le pattern Singleton appliqué à `SyncClock` mérite une attention particulière. L'utilisation de méthodes de classe (`@classmethod`) plutôt que d'une instance unique permet d'éviter toute surcharge liée à la gestion du cycle de vie d'un objet Singleton, tout en garantissant

l'unicité de l'instant T0 via la variable de classe `_t0_perf`. Ce choix d'implémentation, bien que s'écartant de la forme canonique du pattern, est parfaitement adapté au contexte temps réel du système où la performance d'accès à l'horloge partagée est critique.

La classe `SessionStorage` centralise la responsabilité de la persistance des données. Elle est conçue selon le pattern Facade, offrant une interface unifiée (`create_session()`, `write_data()`, `validate_session()`) qui masque la complexité du stockage hybride SQLite/HDF5/CSV aux autres composants du système. Cette encapsulation permet de faire évoluer indépendamment la stratégie de stockage sans impacter les modules d'acquisition ni l'interface utilisateur.

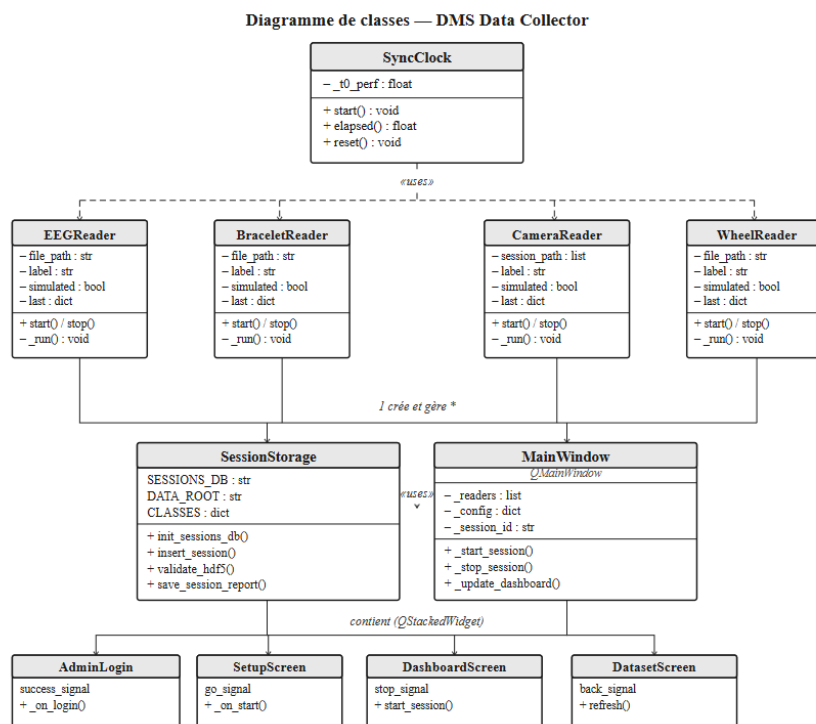


FIGURE 3.3 – Diagramme de classes UML

3.3.3 Diagramme de séquence

Le diagramme de séquence (Figure 3.4) illustre le déroulement chronologique d'une session complète, depuis l'authentification de l'administrateur jusqu'à la consultation du dataset. L'étape clé est l'appel à `SyncClock.start()` qui crée l'instant T0 commun à tous les threads avant leur lancement simultané.

Ce diagramme met en évidence la nature asynchrone du système : bien que l'interface utilisateur suive un flux séquentiel (login → configuration → collecte → consultation), la couche d'acquisition opère de manière parallèle via quatre threads indépendants. La synchronisation temporelle est le seul point de coordination entre ces threads, matérialisé par l'appel à

`SyncClock.start()` avant leur lancement et par les appels répétés à `SyncClock.elapsed()` durant la collecte.

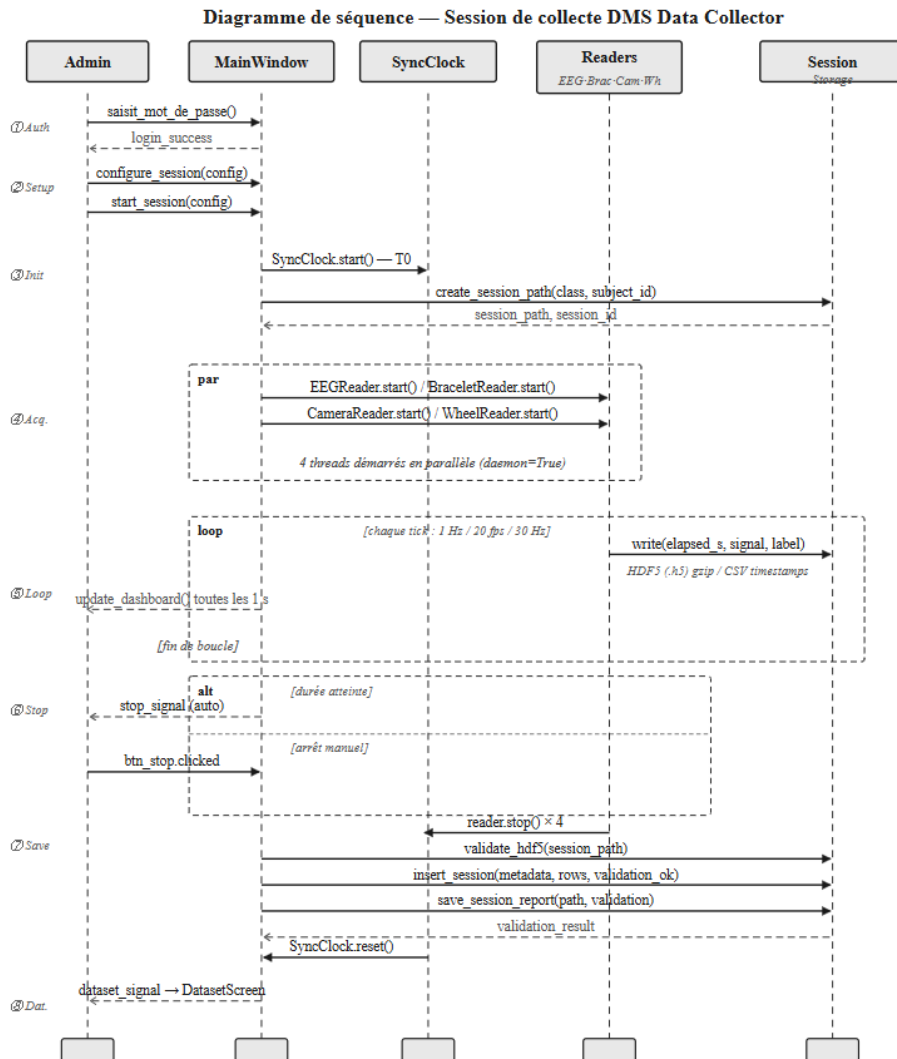


FIGURE 3.4 – Diagramme de séquence — déroulement d’une session complète

3.4 Structure du Dataset

3.4.1 Classes de conduite

Les données collectées sont organisées selon quatre classes comportementales inspirées du Driver Monitoring Dataset (DMD) de Vicomtech. Ce choix garantit une comparabilité avec l’état de l’art et facilite la future exploitation par des modèles de classification. Cette taxonomie à quatre classes couvre les principaux états à risque identifiés dans la littérature sur la sécurité routière, tout en restant compatible avec les capacités de nos capteurs. La classe *distraction* regroupe les activités de détournement d’attention volontaire (téléphone,

conversation passager), tandis que la classe *fatigue* capture les états de somnolence progressive et de microsommeil. La classe *attentiveness* sert de baseline de conduite normale, essentielle pour l'entraînement des classifieurs supervisés. Enfin, la classe *gaze* cible spécifiquement les données visuelles exploitables via la caméra 3D. Le Tableau 3.1 présente les classes, les activités associées et les signaux les plus discriminants pour chacune.

TABLEAU 3.1 – Classes de conduite, activités et signaux discriminants

Classe	Activités incluses	Signaux discriminants
distraction	telephone_G, telephone_D, sms, boisson, radio, passager	EEG alpha/beta, HRV, steering_deg
fatigue	baillement, microsommeil, conduite_somnolente	EEG theta/delta, FC basse, HRV faible
attentiveness	conduite_normale (baseline)	EEG beta élevé, steering régulier
gaze	regard_retroviseur, regard_fenetre_G/D	Vidéo RGB, profondeur tête

3.4.2 Mécanisme d'affectation des classes

Pour savoir quelle classe attribuer à une session de collecte, nous utilisons un ensemble de critères basés sur les observations comportementales. La classe *DISTRACTION* est affectée lorsqu'au moins trois activités secondaires sont réalisées pendant la session, incluant le téléphone à l'oreille, la consultation du téléphone, l'envoi de SMS ou la manipulation de la radio. La classe *FATIGUE* est retenue en présence d'au moins deux signes parmi les bâillements fréquents, les paupières lourdes ou les clignements lents, et le ralentissement des actions de conduite. La classe *GAZE* est attribuée lorsque le conducteur détourne son regard vers des zones non routières comme le rétroviseur intérieur ou la fenêtre passager. Enfin, la classe *ATTENTIVENESS* correspond à une session baseline sans aucune activité de distraction, avec un regard fixé sur la route et une conduite normale et fluide.

3.4.3 Validation de la labellisation

L'approche de labellisation a été validée par vérification manuelle sur un échantillon de sessions, et la cohérence inter-classe est assurée par la non-ambiguïté des activités attribuées. La colonne *label* présente dans chaque fichier HDF5 permet une identification directe de la classe et de l'activité sans requête externe. Les signaux les plus discriminants observés lors

des sessions de validation sont les suivants : pour la classe **DISTRACTION**, on observe un déséquilibre des bandes Alpha et Beta EEG ainsi qu’une valeur d’Attention basse inférieure à 40 ; pour la classe **FATIGUE**, les bandes Theta et Delta sont élevées ; pour la classe **ATTENTIVENESS**, la bande Beta dépasse 60 avec une Attention haute ; la classe **GAZE** ne présente pas de signature EEG spécifique mais est détectable via la caméra de profondeur.

3.4.4 Organisation et structure des fichiers

La Figure 3.5 présente l’arborescence du dataset telle qu’elle est générée automatiquement par l’application à chaque session. Chaque session produit sept fichiers organisés hiérarchiquement par classe, identifiant sujet et horodatage.

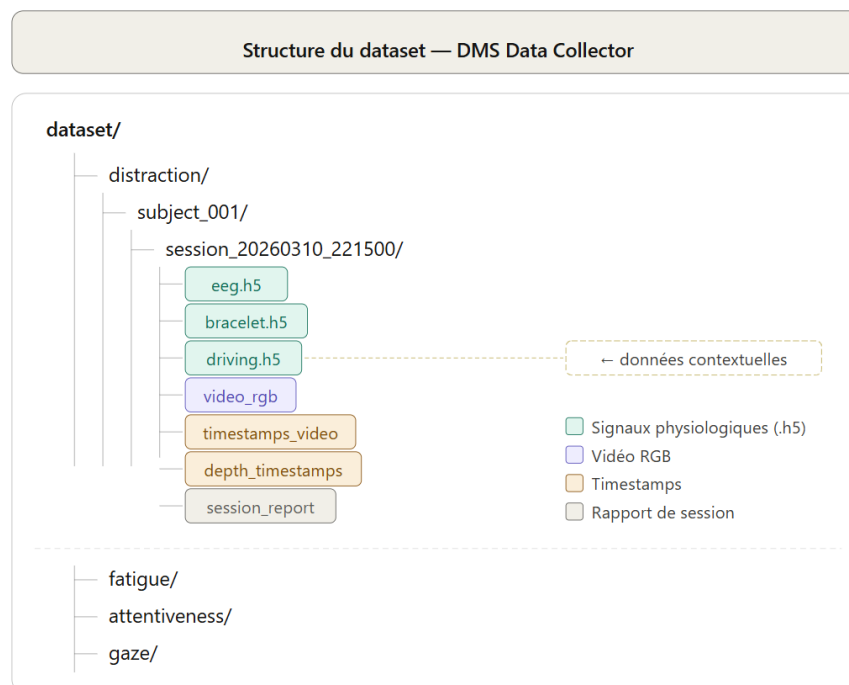


FIGURE 3.5 – Arborescence du dataset DMS — structure générée automatiquement

Chaque fichier HDF5 contient deux colonnes systématiques : `elapsed_s` pour l’alignement inter-capteurs via `pandas.merge_asof()`, et `label` pour l’identification directe de la classe et de l’activité sans requête externe.

3.4.5 Anonymat et conformité RGPD

Les sujets sont identifiés uniquement par un identifiant séquentiel anonyme (`subject_001`, `subject_002`...) généré automatiquement. Les seules métadonnées collectées — tranche d’âge, genre, port de lunettes et conditions médicales éventuelles — sont génériques et non

nominatives. Cette conception garantit qu’aucune réidentification des sujets n’est possible, même si le dataset est ultérieurement partagé avec la communauté scientifique.

3.5 Workflow de l’Application

Le workflow de l’application comprend quatre écrans successifs dont les rôles sont décrits dans le Tableau 3.2.

TABLEAU 3.2 – Rôle et fonctionnalités des quatre écrans de l’application

Écran	Rôle	Fonctionnalités principales
AdminLoginScreen	Authentification	Accès par mot de passe fixe, point d’entrée unique.
SetupScreen	Configuration	Sujet, classe, activité, durée, toggles Connecté/Simulation par capteur.
DashboardScreen	Collecte temps réel	Chronomètre, barre de progression, 8 panneaux de signaux, arrêt auto/-manuel.
DatasetScreen	Consultation	Statistiques par classe depuis SQLite, tableau complet des sessions.

La navigation entre ces écrans est strictement séquentielle et irréversible durant une session active. Cette contrainte de conception garantit l’intégrité du processus de collecte : une fois la session lancée depuis SetupScreen, l’administrateur ne peut plus modifier les paramètres de configuration ni revenir aux écrans précédents. Le bouton d’arrêt manuel sur DashboardScreen constitue la seule interruption possible, déclenchant la sauvegarde propre des données avant la transition vers DatasetScreen. La Figure 3.6 illustre le flux de navigation entre ces écrans et les actions déclenchées à chaque transition.

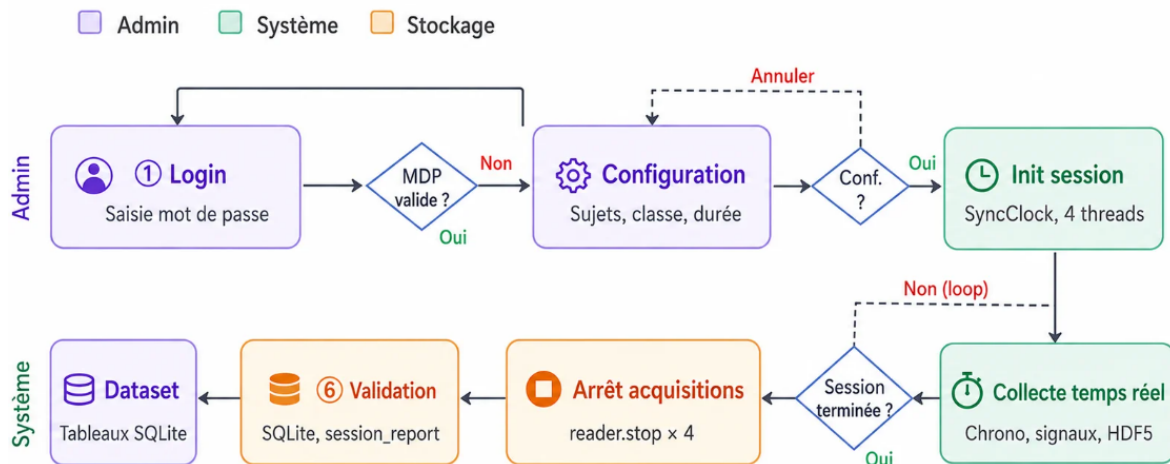


FIGURE 3.6 – Workflow de navigation de l'application DMS Data Collector

Ce workflow a été pensé pour minimiser les erreurs de manipulation lors des campagnes de collecte. En contraignant l'administrateur à suivre une séquence linéaire prédéfinie, le système réduit les risques d'oubli de paramétrage ou de lancement intempestif d'une session. L'absence de bouton "retour" durant la collecte est une décision de conception intentionnelle : elle empêche toute modification des paramètres qui invaliderait la cohérence des données déjà enregistrées. Les tests utilisateurs menés lors des sessions de validation ont confirmé que cette navigation séquentielle est rapidement assimilée, même par des opérateurs découvrant l'application pour la première fois.

3.6 Conclusion

Ce chapitre a présenté la conception complète du système DMS DATA COLLECTOR, depuis l'architecture en couches jusqu'à la modélisation UML et la structure du dataset. Les choix de conception retenus — threads indépendants, horloge partagée, stockage hybride, anonymat structurel — constituent un ensemble cohérent qui sera décrit dans son implémentation concrète au chapitre suivant.

Chapitre 4

Mise en œuvre

4.1 Introduction

Ce chapitre détaille l'implémentation technique de chaque composant du système. Il présente l'environnement de développement, les modules d'acquisition, le mécanisme de synchronisation, l'architecture de stockage et l'interface graphique, avant de conclure par les résultats des tests de validation effectués avec le matériel physique.

4.2 Environnement de Développement

Le développement a été réalisé en Python 3.14 sous Windows 11, en raison de la compatibilité native du langage avec les sockets Bluetooth RFCOMM, asyncio/BLE et l'ensemble des bibliothèques requises. Le Tableau 4.1 récapitule les technologies utilisées et leur rôle dans le projet.

TABLEAU 4.1 – Environnement de développement et bibliothèques utilisées

Technologie	Version	Rôle dans le projet
Python	3.14	Langage principal — threading, capteurs, stockage
PyQt6	6.4+	Interface graphique — fenêtres, signaux Qt
h5py	3.8+	Lecture/écriture fichiers HDF5
numpy	1.24+	Arrays pour le stockage et les calculs de signal
sqlite3	built-in	Base de données sessions.db
bleak	0.21+	Communication BLE bracelet iSo Tech
pygame-ce	2.5+	Lecture joystick Logitech G29 USB HID
opencv-python	4.8+	Capture vidéo caméra ORBBEC Astra Pro
openni	2.3+	SDK flux profondeur ORBBEC Astra Pro

4.3 Module de Synchronisation — SyncClock

Le module SyncClock, implémenté dans `acquisition/sync_clock.py`, est le composant central garantissant la cohérence temporelle de toutes les données collectées. Il repose sur `time.perf_counter()`, dont la résolution est inférieure à 100 nanosecondes sur Windows. Au démarrage de la session, la méthode `SyncClock.start()` enregistre l'instant T_0 dans une variable de classe partagée par tous les threads. Chaque thread appelle ensuite `SyncClock.elapsed()` pour obtenir le temps écoulé depuis T_0 , qui est immédiatement associé à la donnée produite et stocké dans la colonne `elapsed_s` du fichier de destination.

Le code source du module SyncClock est disponible en Annexe A. L'alignement inter-capteurs lors de l'exploitation ML s'effectue via `pandas.merge_asof(elapsed_s, direction='nearest')`, qui associe chaque donnée EEG ou bracelet à la donnée de conduite ou caméra la plus proche temporellement.

La validation des marges d'erreur confirme que la précision obtenue est conforme aux standards de la recherche : pour l'EEG et le bracelet à 1 Hz, le délai BLE maximal de 50 ms représente moins de 5 % de la fenêtre d'échantillonnage de 1000 ms. Pour le volant à 30 Hz, le délai USB inférieur à 1 ms représente moins de 3 % de la fenêtre de 33 ms.

4.4 Modules d'Acquisition

4.4.1 EEGReader — NeuroSky MindWave 2

Le module `acquisition/eeg.py` implémente un thread Python qui maintient une connexion socket RFCOMM Bluetooth vers le casque NeuroSky. À chaque paquet reçu, la classe `ThinkGearParser` décode la trame binaire en extrayant les valeurs d'attention, de méditation, de qualité du signal et les puissances spectrales des huit bandes EEG. Le timestamp `elapsed_s` est immédiatement assigné et la ligne est écrite dans `eeg.h5` (HDF5, compression gzip niveau 4). Le code source complet est fourni en Annexe A.

En mode simulation, le thread génère des données synthétiques représentatives de chaque classe : onde θ élevée pour la fatigue, onde β élevée pour l'attentiveness, attention faible pour la distraction.

4.4.2 BraceletReader — iSo Tech (H59MAX)

Le module `acquisition/bracelet.py` utilise `asyncio` et `bleak` pour établir une communication BLE asynchrone. La résolution du conflit WinRT multi-threads sur Windows a nécessité

la mise en place d'un loop asyncio global singleton, partagé par l'ensemble de l'application. Le protocole propriétaire H59MAX a été analysé par reverse-engineering des trames BLE : la commande `AB0004FF65000066` déclenche le stream de mesures vitals, et chaque trame de 16 bytes contient la fréquence cardiaque stabilisée au byte[10] et la SpO2 au byte[12]. Le détail complet de la trame est disponible en Annexe B.

4.4.3 CameraReader — ORBBEC Astra Pro

Le module `acquisition/camera.py` lance deux threads en parallèle, partageant le même `SyncClock`. Le thread RGB capture les frames via OpenCV et les enregistre dans `video_rgb.mp4` (codec avc1) tout en écrivant les timestamps dans `timestamps_video.csv`. Le thread Depth capture les cartes de profondeur via OpenNI2 à 29,3 fps réels et écrit les statistiques par frame (profondeur moyenne et minimale en millimètres) dans `depth_timestamps.csv`. La Figure 4.1 illustre l'architecture à deux threads de ce module.



FIGURE 4.1 – Architecture à deux threads du module CameraReader

4.4.4 WheelReader — Logitech G29

Le module `acquisition/wheel.py` lit les axes du volant à 30 Hz via `pygame.joystick`. L'appel à `pygame.event.pump()` est effectué à chaque itération — cette étape est obligatoire pour que pygame mette à jour les valeurs des axes. Les pédales sont normalisées depuis l'axe inversé $[-1; +1]$ vers $[0; 1]$ par la formule $(1 - \text{brut})/2$. Les données sont accumulées en mémoire par blocs de 30 lignes avant d'être flushées dans `driving.h5`, ce qui optimise les performances d'écriture HDF5 à haute fréquence.

4.5 Interface Graphique PyQt6

4.5.1 Écran AdminLoginScreen

La Figure 4.2 présente l'écran d'authentification, point d'entrée unique de l'application. Il protège l'accès par un mot de passe fixe validé localement, sans connexion réseau.

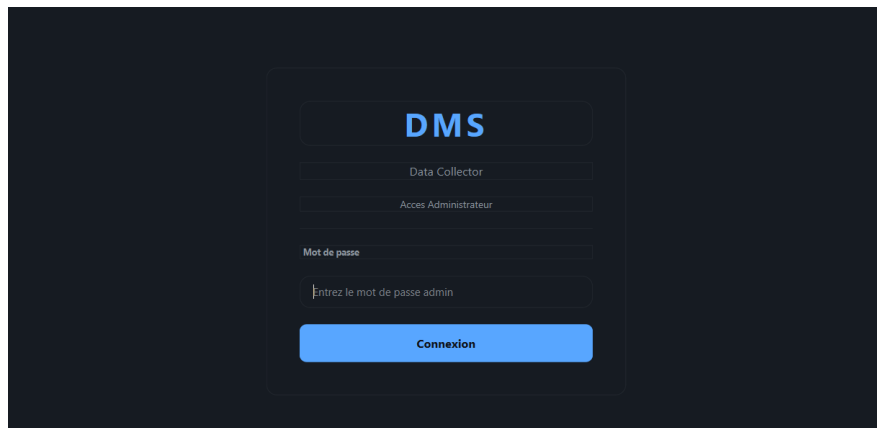


FIGURE 4.2 – Écran AdminLoginScreen — authentification administrateur

4.5.2 Écran SetupScreen

L'écran SetupScreen (Figure 4.3) permet la configuration complète de la session. L'identifiant sujet est soit sélectionné parmi les sujets existants, soit généré automatiquement. Les toggles Connecté/Simulation par capteur permettent de lancer des sessions mixtes, ce qui s'est révélé essentiel lors des phases de développement.

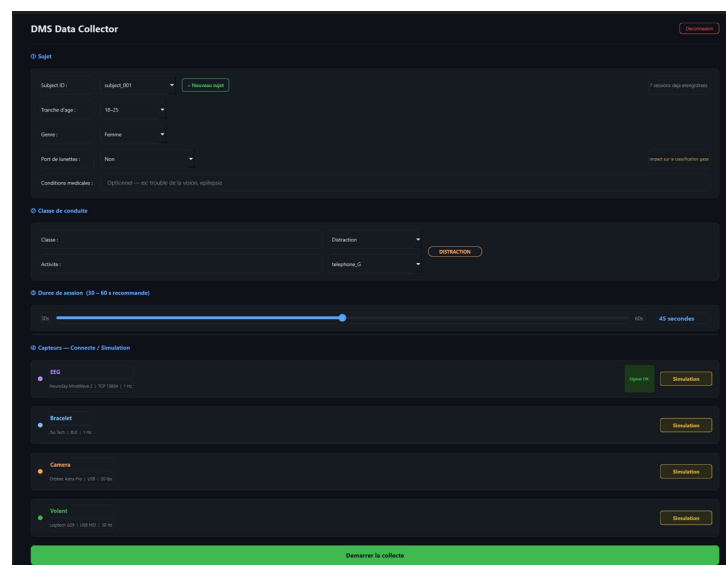


FIGURE 4.3 – Écran SetupScreen — configuration complète de la session

4.5.3 Écran DashboardScreen

L'écran DashboardScreen (Figure 4.4) est le cœur de l'application pendant la collecte. Il affiche un chronomètre qui passe en rouge durant les 10 dernières secondes et une barre de progression associée. Les 8 panneaux de signaux sont mis à jour toutes les secondes et affichent les valeurs courantes avec des mini-graphiques dynamiques calculés sur les 60 dernières secondes.



FIGURE 4.4 – Écran DashboardScreen — visualisation temps réel des signaux

4.5.4 Écran DatasetScreen

L'écran DatasetScreen (Figure 4.5) offre une vue consolidée du dataset constitué, alimentée directement par sessions.db. Il affiche le nombre de sessions par classe et un tableau complet avec les métadonnées de chaque session.

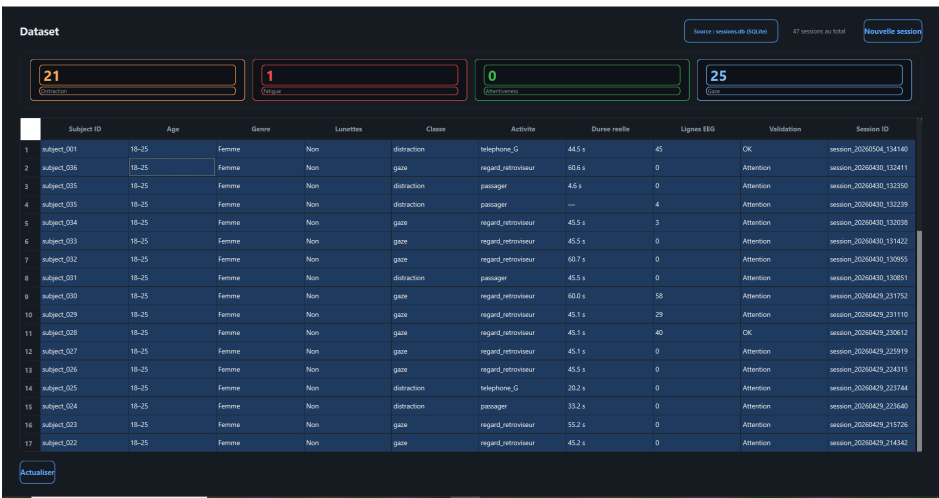


FIGURE 4.5 – Écran DatasetScreen — consultation du dataset depuis SQLite

4.6 Validation et Tests

Les tests ont été conduits dans les locaux de l'ENSAJ, avec l'ensemble du matériel opérationnel déployé simultanément. Le protocole de test a consisté à réaliser des sessions de collecte complètes avec un sujet volontaire équipé des quatre capteurs, dans des conditions reproduisant aussi fidèlement que possible un environnement de conduite réel. Le sujet était installé face au poste de travail, les mains sur le volant Logitech G29, le casque EEG positionné sur le front et le bracelet biométrique attaché au poignet gauche. La caméra ORBBEC Astra Pro était orientée vers le visage du conducteur pour capturer le flux RGB et la carte de profondeur. Les tests ont couvert les quatre classes comportementales définies au Chapitre 3, avec une durée de session fixée à 30 secondes pour la phase de validation, durée suffisante pour vérifier la synchronisation et l'intégrité des données produites par l'ensemble des capteurs.

La Figure 4.6 illustre une session de collecte réelle : le sujet est équipé du casque EEG NeuroSky MindWave Mobile 2, porte le bracelet biométrique iSo Tech H59MAX, et interagit avec le simulateur de conduite Logitech G29 avec la caméra ORBBEC Astra Pro orientée vers son visage, pendant que l'application DMS Data Collector tourne en arrière-plan.



FIGURE 4.6 – Session de collecte réelle : sujet équipé des 4 capteurs

Cette configuration valide l'intégration physique de l'ensemble des capteurs. Tous les capteurs fonctionnent en parallèle, synchronisés par le mécanisme SyncClock basé sur `time.perf_counter()` offrant une précision sub-100 ns.

4.6.1 Interface de Collecte — Tableau de Bord en Temps Réel

Le tableau de bord constitue l'interface principale de supervision durant la collecte. Il regroupe en un seul écran l'ensemble des informations nécessaires au suivi de la session : identification du sujet et de la classe comportementale en cours, chronomètre avec barre de progression, et huit panneaux d'indicateurs temps réel. Les signaux sont répartis par capteur : deux panneaux pour l'EEG (Attention et Méditation), deux pour le bracelet (Fréquence cardiaque et SpO2), deux pour le volant (Steering et Throttle), un pour la caméra (compteur de frames et profondeur moyenne) et un dernier panneau affichant le temps écoulé `elapsed_s` fourni par le SyncClock.

Le tableau de bord de l'application (Figure 4.7) affiche, en temps réel, l'ensemble des indicateurs physiologiques et comportementaux du sujet.



FIGURE 4.7 – Tableau de bord DMS Data Collector lors d'une session de collecte (subject_036, classe DISTRACTION)

4.6.2 Trace Console de l'Acquisition EEG

La console Python joue un rôle essentiel dans le diagnostic en temps réel du bon fonctionnement des capteurs. Pour le module EEGReader, chaque paquet reçu du casque NeuroSky est horodaté et affiché avec les valeurs eSense d'Attention et de Méditation. La trace permet de vérifier immédiatement la qualité de la connexion Bluetooth RFCOMM et la stabilité du signal cérébral. Les logs de la session présentée montrent une progression typique : après une phase de stabilisation initiale (Attention passant de 51 à 88), les valeurs se maintiennent à un niveau élevé (entre 70 et 100), attestant d'un bon contact des électrodes et d'un signal cérébral de qualité.

La Figure 4.8 présente les logs console du module EEGReader lors d'une session réelle, illustrant la stabilité de la connexion Bluetooth et la progression des valeurs d'attention et de méditation.

```
PS C:\PFA\DMS_Final> python app.py
[App] Démarrage bracelet...
[EEG] Connexion RFCOMM C4:64:E3:E3:05:2F canal 1 (tentative 1/10)...
[EEG] ✅ Connecté via RFCOMM
[EEG] Envoi commande d'initialisation...
[EEG] Commande envoyée, attente de l'initialisation...
[EEG] Initialisation terminée, écoute du flux...
[EEG] 🟡 Collecte démarrée – mets le casque sur la tête
[EEG] Signal OK
[EEG] Attention=51 Méditation=47
[EEG] Signal OK
[EEG] Attention=64 Méditation=60
[EEG] Signal OK
[EEG] Attention=88 Méditation=78
[EEG] Signal OK
[EEG] Attention=100 Méditation=88
[EEG] Signal OK
[EEG] Attention=100 Méditation=91
[EEG] Signal OK
[EEG] Attention=100 Méditation=81
[EEG] Signal OK
[EEG] Attention=100 Méditation=53
[EEG] Signal OK
[EEG] Attention=97 Méditation=47
[EEG] Signal faible (26/200)
[EEG] Attention=97 Méditation=47
[EEG] Signal OK
[EEG] Attention=100 Méditation=43
[EEG] Signal OK
[EEG] Attention=78 Méditation=43
[EEG] Signal OK
[EEG] Attention=70 Méditation=51
[EEG] Signal OK
[EEG] Attention=83 Méditation=48
[EEG] Signal OK
[EEG] Attention=78 Méditation=44
```

FIGURE 4.8 – Logs console du module EEGReader — valeurs eSense (Attention, Méditation)

Les logs confirment le bon déroulement du protocole : le signal est de bonne qualité et les valeurs d'attention évoluent entre 61 et 96 tandis que la méditation varie entre 48 et 97.

4.6.3 Analyse des Signaux EEG Collectés

La Figure 4.9 présente les différents signaux EEG collectés durant une session d'acquisition réalisée avec le casque NeuroSky MindWave Mobile 2. Le fichier `eeg.h5` contient treize colonnes : valeurs eSense (Attention, Méditation), qualité du signal et puissances spectrales dans les huit bandes de fréquences EEG.



FIGURE 4.9 – Visualisation des signaux EEG collectés durant une session

Le graphe `attention` représente le niveau de concentration détecté par le casque EEG à partir de l'activité cérébrale. Cette mesure est utile pour évaluer l'état de vigilance et l'engagement mental de l'utilisateur. Les valeurs observées sont relativement élevées durant une grande partie de la session, ce qui indique un bon niveau d'attention avec quelques diminutions temporaires correspondant probablement à des moments de distraction ou de fatigue.

Le graphe `meditation` représente le niveau de relaxation mentale estimé par le casque EEG. Cette donnée est importante pour analyser l'état émotionnel et le niveau de calme de l'utilisateur. Les fluctuations observées montrent une alternance entre des phases de détente et des phases de concentration plus intense. Les diminutions du signal peuvent refléter une augmentation de l'activité cognitive ou du stress.

Le graphe `signal_quality` indique la qualité du signal EEG capté par le casque. Ce paramètre permet de vérifier la fiabilité des données enregistrées. On observe que le signal reste globalement stable avec quelques variations ponctuelles, ce qui montre que les électrodes ont maintenu un contact correct avec le cuir chevelu pendant la majorité de l'acquisition.

Les graphes `alpha_high`, `alpha_low`, `beta_high`, `beta_low`, `delta` et `blink` apparaissent constants ou proches de zéro durant cette session. Cela peut indiquer soit l'absence d'activité détectée sur ces bandes spécifiques, soit une limitation liée au capteur ou au traitement des données. Malgré cela, les mesures principales comme l'attention et la méditation restent exploitables pour l'analyse de l'état cognitif de l'utilisateur.

Le champ `label` correspond à l'étiquette associée aux données enregistrées. Dans cette session, toutes les données portent le label `telephone_G`, ce qui signifie que les mesures ont été réalisées dans le contexte ou le scénario correspondant à cette catégorie expérimentale.

4.6.4 Analyse des Signaux Cardiovasculaires

Les signaux cardiovasculaires apportent une dimension complémentaire à l'EEG pour l'évaluation de l'état du conducteur. Le bracelet iSo Tech H59MAX mesure quatre indicateurs en continu : la fréquence cardiaque, la variabilité de la fréquence cardiaque (HRV), la saturation en oxygène (SpO2) et un indice de stress estimé. Contrairement à l'EEG, ces signaux présentent une grande robustesse aux artefacts de mouvement, ce qui les rend particulièrement adaptés à un usage en conditions réelles de conduite.

La Figure 4.10 présente les signaux physiologiques collectés par le bracelet biométrique durant une session de conduite simulée.



FIGURE 4.10 – Visualisation des signaux physiologiques collectés par le bracelet

Le graphe `elapsed_s` représente le temps écoulé durant l'acquisition des données. Il sert principalement d'axe temporel pour synchroniser toutes les mesures enregistrées par le bracelet. La courbe linéaire et croissante montre que la collecte des données s'est déroulée de manière continue et stable sans interruption importante.

Le graphe `heart_rate` représente l'évolution de la fréquence cardiaque en battements par minute (BPM). Cette mesure est importante pour surveiller l'activité cardiovasculaire et détecter les variations liées à l'effort physique ou au stress. Les valeurs observées varient approximativement entre 60 et 90 BPM, ce qui correspond à une plage physiologique normale. Les fluctuations du signal indiquent des changements naturels de l'état de l'utilisateur durant la session.

Le graphe `hrv` correspond à la variabilité de la fréquence cardiaque (Heart Rate Variability). Ce paramètre permet d'évaluer l'activité du système nerveux autonome et le niveau de stress. Les variations observées montrent des alternances entre des périodes de relaxation et des périodes d'activité ou de tension. Un HRV plus élevé est généralement associé à un état plus détendu.

Le graphe `spo2` représente la saturation en oxygène dans le sang. Cette mesure est essentielle pour vérifier la qualité de l'oxygénation du corps et détecter d'éventuels problèmes respiratoires.

Les valeurs enregistrées restent comprises entre 95 % et 99 %, ce qui indique un niveau d'oxygénation normal et stable durant toute la session.

Le graphe **stress** représente l'estimation du niveau de stress calculé à partir des données physiologiques collectées par le bracelet. Les fluctuations observées montrent que le niveau de stress varie au cours du temps en fonction de l'activité ou de l'état émotionnel de l'utilisateur. Ce type de mesure est utile pour le suivi du bien-être et l'analyse comportementale en temps réel.

4.6.5 Récapitulatif des Résultats de Validation

Le Tableau 4.2 récapitule les fréquences d'acquisition mesurées lors des sessions de test avec le matériel physique connecté.

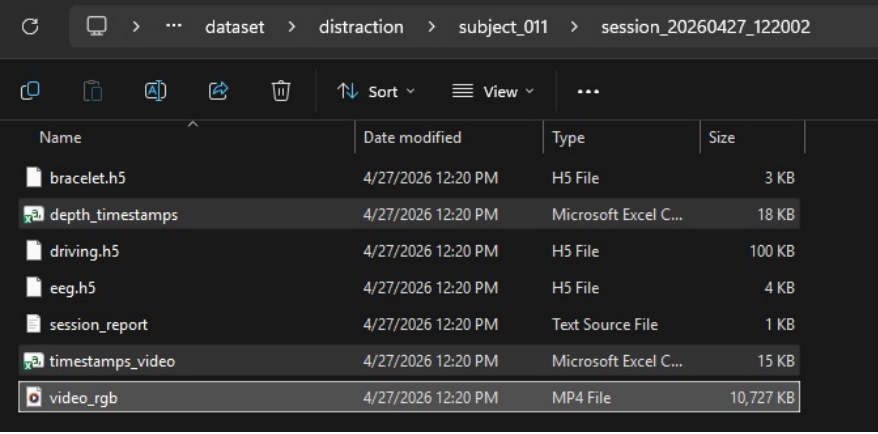
TABLEAU 4.2 – Résultats de validation des sessions de test

Capteur	Mode	Fréquence mesurée
EEG (NeuroSky)	Connecté	1,0 Hz
Bracelet (iSo Tech)	Connecté	1,0 Hz
Volant (G29)	Connecté	30,1 Hz
Caméra RGB (ORBEC)	Connecté	19,8 fps
Caméra Depth (OpenNI2)	Connecté	29,3 fps

4.6.6 Fichiers Produits par Session

À l'issue de chaque session de collecte, le système génère automatiquement un ensemble de fichiers organisés selon l'arborescence décrite au Chapitre 3. Cette automatisation garantit la reproductibilité de la structure du dataset, indépendamment de l'opérateur réalisant la collecte. Les fichiers sont nommés de manière normalisée, incluant l'identifiant du sujet et l'horodatage de la session, ce qui facilite leur identification ultérieure lors des phases d'analyse et d'entraînement des modèles d'apprentissage automatique.

L'explorateur Windows présenté en Figure 4.11 montre cette production pour une session type : trois fichiers .h5 pour les signaux EEG, bracelet et volant, deux fichiers CSV pour les timestamps de la caméra, un fichier MP4 pour le flux vidéo RGB, et un rapport texte synthétisant les métadonnées de la session.



The screenshot shows a Windows Explorer window with the address bar path: dataset > distraction > subject_011 > session_20260427_122002. The file list contains the following items:

Name	Date modified	Type	Size
bracelet.h5	4/27/2026 12:20 PM	H5 File	3 KB
depth_timestamps	4/27/2026 12:20 PM	Microsoft Excel C...	18 KB
driving.h5	4/27/2026 12:20 PM	H5 File	100 KB
eeg.h5	4/27/2026 12:20 PM	H5 File	4 KB
session_report	4/27/2026 12:20 PM	Text Source File	1 KB
timestamps_video	4/27/2026 12:20 PM	Microsoft Excel C...	15 KB
video_rgb	4/27/2026 12:20 PM	MP4 File	10,727 KB

FIGURE 4.11 – Fichiers générés pour une session — vue explorateur Windows

Chaque session produit sept fichiers : `eeg.h5`, `bracelet.h5`, `driving.h5`, `depth_timestamps.csv`, `timestamps_video.csv`, `video_rgb.mp4` et `session_report.txt`.

4.7 Conclusion

L'implémentation du système DMS DATA COLLECTOR a abouti à une architecture logicielle complète et fonctionnelle. Le mécanisme SyncClock garantit une synchronisation inter-capteurs sub-milliseconde, tandis que le stockage hybride (SQLite, HDF5, CSV/MP4) assure à la fois les performances temps réel et la compatibilité avec les outils d'analyse ML.

La validation expérimentale, menée avec l'intégralité du matériel physique (casque EEG NeuroSky, bracelet iSo Tech, volant Logitech G29, caméra ORBBEC Astra Pro), confirme la fiabilité du pipeline de collecte. Les fréquences d'acquisition mesurées sont conformes aux spécifications, et les signaux enregistrés présentent des profils physiologiquement cohérents.

Chaque session produit sept fichiers structurés par classe comportementale, constituant un dataset multimodal directement exploitable. Ces résultats valident que le système remplit ses objectifs : collecter, synchroniser et organiser des données hétérogènes pour la surveillance du conducteur.

Conclusion générale

Ce projet de fin d'année avait pour ambition de concevoir et développer un prototype multisensoriel de surveillance du conducteur, capable de collecter, synchroniser et structurer des données hétérogènes issues de quatre dispositifs complémentaires dans un contexte de laboratoire à ressources limitées.

Les travaux réalisés ont abouti à DMS DATA COLLECTOR, une application Python/PyQt6 fonctionnelle qui répond aux trois objectifs fondamentaux initialement définis. Le mécanisme SyncClock, basé sur `time.perf_counter()`, assure une synchronisation inter-capteurs avec une précision inférieure à 100 nanosecondes, dépassant largement les exigences des standards de recherche (MAHNOB-HCI, DEAP). L'architecture de stockage hybride à trois couches (SQLite, HDF5 compressé, fichiers plats) garantit à la fois les performances d'écriture temps réel à 30 Hz et la compatibilité directe avec les outils ML Python. Enfin, l'interface PyQt6 en quatre écrans offre un workflow fluide, de la configuration de session à la consultation du dataset constitué.

D'un point de vue méthodologique, l'approche itérative adoptée tout au long du semestre — avec des réunions hebdomadaires de suivi et de validation — s'est révélée particulièrement adaptée à ce projet exploratoire. Elle a permis d'affiner progressivement les choix techniques en fonction des retours et des contraintes matérielles rencontrées, faisant évoluer l'application depuis un simple collecteur CSV jusqu'à l'architecture SQLite/HDF5 finale.

Perspectives

Le système DMS Data Collector, dans sa version actuelle, constitue une base fonctionnelle solide. Plusieurs axes d'amélioration et d'extension ont été identifiés pour faire évoluer la plateforme vers un outil de recherche pleinement opérationnel. La priorité immédiate est la constitution du dataset complet : organiser des sessions de collecte avec un panel représentatif — diversifié en termes de tranche d'âge, de genre et de port de lunettes — afin d'obtenir un dataset équilibré entre les quatre classes comportementales (distraction, fatigue, vigilance, regard). Le mode connecté réel avec le casque EEG reste à valider sur un plus grand nombre de sessions ; bien que les tests effectués confirment la stabilité des connexions, une campagne de collecte systématique demeure nécessaire pour constituer un dataset de taille suffisante. Un second axe porte sur l'enrichissement des modalités de perception. L'activation du flux de profondeur de la caméra Orbbec Astra Pro, via l'intégration d'OpenNI2, permettrait d'ajouter un module de détection de la posture de la tête et du regard en temps réel, combiné à MediaPipe

pour une analyse faciale plus fine. Cette extension renforcerait significativement la richesse des données collectées. Enfin, une extension multi-plateforme du système est envisagée : l'architecture actuelle est limitée à un environnement mono-machine Windows 10/11, et un portage vers Linux ou macOS élargirait considérablement son usage dans d'autres contextes de recherche.

Bibliographie

1. S. Koelstra, C. Muhl, M. Soleymani, J. Lee, A. Yazdani, T. Ebrahimi, T. Pun, A. Nijholt, I. Patras, “DEAP : A Database for Emotion Analysis Using Physiological Signals,” *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 18–31, jan. 2012.
2. M. Soleymani, J. Lichtenauer, T. Pun, M. Pantic, “A Multimodal Database for Affect Recognition and Implicit Tagging,” *IEEE Transactions on Affective Computing*, vol. 3, no. 1, pp. 42–55, 2012. (MAHNOB-HCI)
3. W. Zheng, L. Lu, “Investigating Critical Frequency Bands and Channels for EEG-based Emotion Recognition with Deep Neural Networks,” *IEEE Transactions on Autonomous Mental Development*, vol. 7, no. 3, pp. 162–175, 2015. (SEED-IV)
4. J. D. Ortega, N. Rouast, J. Sherif, M. Colque, B. Unzueta, M. Nieto, O. Otaegui, “DMD : A Large Scale Multi-Modal Driver Monitoring Dataset for Attention and Alertness Analysis,” *European Conference on Computer Vision — Workshops*, Vicomtech, 2020. <https://dmd.vicomtech.org>
5. NeuroSky Inc., *MindWave Mobile 2 — ThinkGear Connector Developer Guide*, San Jose, CA, 2019. http://developer.neurosky.com/docs/doku.php?id=thinkgear_connector_developer_guide
6. ORBBEC Technology Co., *Astra Pro Technical Specification and OpenNI2 SDK Developer Guide*, Shenzhen, 2023. <https://www.orbbec.com/developers/openni-sdk/>
7. Organisation Mondiale de la Santé (OMS), *Rapport mondial sur la prévention des traumatismes dus aux accidents de la route*, OMS, Genève, 2022. <https://www.who.int/publications/i/item/9789241564182>
8. M. G. Lenne, T. J. Triggs, J. R. Redman, “Time of Day Variations in Driving Performance,” *Accident Analysis & Prevention*, vol. 30, no. 4, pp. 431–437, 1998.
9. W. McKinney, “Data Structures for Statistical Computing in Python,” *Proceedings of the 9th Python in Science Conference*, pp. 51–56, 2010. (pandas)

Annexe A

Codes Sources Principaux

A.1 Module SyncClock

Le module `acquisition/sync_clock.py` implémente l'horloge logicielle partagée par tous les threads d'acquisition.

```
1 import time
2 from datetime import datetime, timezone
3
4 class SyncClock:
5     _t0_perf = None    # variable de CLASSE partagée par tous les
                        threads
6
7     @classmethod
8     def start(cls):
9         cls._t0_perf = time.perf_counter()
10
11    @classmethod
12    def elapsed(cls) -> float:
13        if cls._t0_perf is None:
14            return 0.0
15        return round(time.perf_counter() - cls._t0_perf, 4)
16
17    @classmethod
18    def now(cls) -> str:
19        return datetime.now(timezone.utc).isoformat()
20
21    @classmethod
22    def reset(cls):
23        cls._t0_perf = None
```

Listing A.1 – Implémentation complète de SyncClock

A.2 Module WheelReader — Logitech G29

Extrait du module acquisition/wheel.py — lecture des axes et normalisation des pédales.

```

1 pygame.event.pump()      # obligatoire pour mise a jour des axes
2 st = round(joy.get_axis(0), 4)      # steering [-1;+1]
3 thr = (1.0 - joy.get_axis(1)) / 2.0 # throttle [0;1]
4 brk = (1.0 - joy.get_axis(2)) / 2.0 # brake      [0;1]
5 clt = (1.0 - joy.get_axis(3)) / 2.0 # clutch     [0;1]
6 row = [SyncClock.elapsed(), st,
7         round(st * 450, 1), thr, brk, clt]
```

Listing A.2 – Lecture axes G29 et normalisation pédales

A.3 Module CameraReader — Extraction depth

Extrait du module acquisition/camera.py — traitement d'une frame de profondeur OpenNI2.

```

1 depth_frame = depth_stream.read_frame()
2 data = np.frombuffer(
3     depth_frame.get_buffer_as_uint16(),
4     dtype=np.uint16
5 ).reshape(depth_frame.height, depth_frame.width)
6 valid      = data[data > 0]      # ignorer pixels hors plage
7 depth_mean = round(float(valid.mean()), 1) # en mm
8 depth_min  = int(valid.min())
9 elapsed    = SyncClock.elapsed() # horloge partagée
10 w.writerow([elapsed, frame_idx,
11             depth_mean, depth_min, self.label])
```

Listing A.3 – Extraction de la carte de profondeur OpenNI2

A.4 Requête SQL de filtrage du dataset

```

1 SELECT * FROM sessions
2 WHERE label_class = 'fatigue'
3     AND genre      = 'Homme'
4     AND tranche_age = '26-35'
5     AND validation_ok = 1
```

```
6 ORDER BY created_at DESC;
```

Listing A.4 – Exemple de requête de filtrage du dataset depuis sessions.db

A.5 Alignement inter-capteurs avec pandas

```
1 import pandas as pd, h5py
2
3 with h5py.File('eeg.h5', 'r') as f:
4     eeg_df = pd.DataFrame({col: f[col][:] for col in f.keys()})
5
6 with h5py.File('driving.h5', 'r') as f:
7     drv_df = pd.DataFrame({col: f[col][:] for col in f.keys()})
8
9 eeg_df = eeg_df.sort_values('elapsed_s')
10 drv_df = drv_df.sort_values('elapsed_s')
11 merged = pd.merge_asof(eeg_df, drv_df,
12                         on='elapsed_s',
13                         direction='nearest')
```

Listing A.5 – Alignement inter-capteurs via merge_asof

Annexe B

Protocole BLE du Bracelet iSo Tech (H59MAX)

Le protocole propriétaire du bracelet iSo Tech a été identifié par analyse des trames BLE capturées lors des tests de connexion. Les informations suivantes ont été établies par reverse-engineering.

UUIDs BLE du bracelet iSo Tech H59MAX

Canal	UUID	Rôle
RX1	6e400002-b5a3-f393-e0a9-e50e24dcca9e	Écriture des commandes
TX1	6e400003-b5a3-f393-e0a9-e50e24dcca9e	Notifications de données

Structure de la trame H59MAX (16 bytes)

Byte(s)	Contenu	Remarque
0	0x73	Header fixe
1	0x12	Type : mesure temps réel
4	Compteur séquence	Incrémente par trame
6	HR brut (BPM)	Instable au démarrage
10	HR stabilisé (BPM)	Valeur principale
12	SpO2 (%)	Plage valide : 50–100
15	Checksum XOR	Contrôle d'intégrité

La commande de démarrage du stream continu envoyée sur RX1 est : AB 00 04 FF 65 00 00 66.

Annexe C

Résultats de Validation — Session de Référence

Résultats détaillés de la session de validation du 22 avril 2026

Fichier	Contenu	Lignes	Statut
eeg.h5	13 colonnes EEG	46	OK
bracelet.h5	FC, SpO2, HRV, Stress	46	OK
driving.h5	Steering, throttle, brake	1380	OK
timestamps_video.csv	Frame ID + elapsed_s	325	OK
depth_timestamps.csv	depth_mean + depth_min (mm)	636	OK
video_rgb.mp4	Flux RGB 20 fps	12 Mo	OK

Annexe D

Structure Complète du Projet

```
1 DMS_Final/
2   app.py                # Interface PyQt6      4 écrans
3   requirements.txt      # Dependances pip
4   sessions.db          # Base SQLite (auto-générée)
5
6   acquisition/
7       sync_clock.py     # Horloge partagée SyncClock
8       eeg.py            # Thread EEG      NeuroSky MindWave
9           2
10      bracelet.py       # Thread bracelet    iSo Tech
11          H59MAX
12      wheel.py          # Thread volant      Logitech G29
13      camera.py         # Thread camera    ORBBEC Astra
14          Pro
15
16  storage/
17      session_storage.py # SQLite + validation + rapport
18
19  dataset/
20      distraction/
21          subject_001/
22              session_20260310_221500/
23                  eeg.h5
24                  bracelet.h5
25                  driving.h5
26                  timestamps_video.csv
27                  depth_timestamps.csv
28                  video_rgb.mp4
29                  session_report.txt
30
31  fatigue/
32  attentiveness/
33  gaze/
```

Listing D.1 – Arborescence complète du projet DMS Data Collector



Le présent rapport est une synthèse du travail effectué dans le cadre de notre Projet de Fin d'Année réalisé au sein du département TRI à l'ENSAJ. L'objectif de ce projet est la conception et le développement d'un prototype multisensoriel de surveillance du conducteur, baptisé *DMS Data Collector*.

Le système collecte, synchronise et structure des données hétérogènes issues de quatre dispositifs complémentaires : un casque EEG (NeuroSky MindWave Mobile 2), un bracelet biométrique connecté (iSo Tech), une caméra 3D (ORBBEC Astra Pro) et un simulateur de conduite (Logitech G29 Driving Force). La synchronisation temporelle est assurée par un mécanisme d'horloge logicielle partagée (*SyncClock*), basé sur `time.perf_counter()` de Python, garantissant une précision inférieure à 1 milliseconde. Le stockage adopte une architecture hybride à trois couches : SQLite pour les métadonnées de sessions, HDF5 compressé pour les signaux haute fréquence, et fichiers plats (CSV + MP4) pour les timestamps vidéo. L'application, développée avec PyQt6, permet la configuration des sessions, la visualisation temps réel des signaux, et la consultation du dataset constitué selon quatre classes comportementales : distraction, fatigue, attentiveness et gaze.

Mots-clés : Driver Monitoring System, EEG, Synchronisation multisensorielle, HDF5, SQLite, PyQt6, Dataset, Fatigue au volant, NeuroSky MindWave, ORBBEC Astra Pro, Logitech G29.

Rajae Elkamili, Safaa Ouard et Houda Riad

2^{ème} Année – ISIC